

Subiecte pp

PDF_epubPY_JsoupKT

- **Python:** Utilizand secvente din Python (PyFunctional), corutinele(asyncio) si impartirea in mai multe bucati, sa se elimine dintr-un fisier text rezultat din conversia unui epub (aveti asa ceva de la o tema) spatiile multiple, salturile la linie noua. La sfarsit va fi generat noul fisier. Se vor folosi aboradri de tipul impacheteaza-proceseaza-despacheteaza

```
from epub2txt import epub2txt
import asyncio
import re
from functional import seq

async def process_string(deProcesat):
    rezz1 = re.sub(" +", " ", deProcesat)
    rezz2 = seq(list(rezz1)).filter(lambda it: it != "\n").make_string("")
    file2 = open("ebook_modificat.txt",'w')
    file2.write(rezz2)

async def main():
    rezz = epub2txt("sample1.epub")
    await asyncio.gather(process_string(rezz))

if __name__ == '__main__':
    asyncio.run(main())
```

- **Kotlin:** Sa se creeze in Kotlin un program care sa descarce o pagina HTML< o analizeaza cu modulul Jsoup si salveaza separat toate imaginile grafice din el. Se vor cauta tag-uri de tip 'img' si se va extrage valoarea atributului src(sursa imaginii respective), pe baza careia poate fi descarcata imaginea cu cererea GET

```
import org.jsoup.Jsoup
import java.io.File
import java.net.URL

fun main() {
```

```

val url = "https://www.geeksforgeeks.org/html-images/"
val doc = Jsoup.connect(url).get()
var i = 0

val outputDirector = File("Imagini Descarcate")
if (!outputDirector.exists()) {
    outputDirector.mkdir()
}

for (image in doc.select("img")) {
    try {
        var imgLnk = image.attr("abs:src")
        println(" Dowloading: $imgLnk")
        val urlImg = URL(imgLnk).readBytes()
        val imageName = "image_$.jpg"
        File(outputDirector, imageName).writeBytes(urlImg)
        println(" Saved: $imageName")
        i++
    } catch (e: Exception) {
        println(" Nu se poate descarca imaginea: ${e.message}")
    }
}
}

```

PDF_FunctorKT_decoratorPY

- **Python:** Utilizand modelul pod(bridge) sa se scrie in Python un program care va porni de la un model mamifer si apoi sa permita instantiere de obiecte de tip om, femeie, .caine si pisica. Acestor obiecte li se vor adauga 3-4 functionalitati suplimentare la alegere utilizand decoratorii(de exemplu o functie care descrie/permite interactiunea dintre om si femeie sau om si pisica etc). Se vor desena diagrama si clase de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```

def interactiunea_om_pisica(func):
    def wrapper(self, animal):
        if isinstance(animal, Pisica):
            print(f"{self.num} interactiuneaza cu pisica {animal.num}")
        else:
            print(f"{self.num} incearca sa interactiuneze cu un animal incompatibil.")
        return wrapper

```

```
def interactiune_pisica_caine(func):
    def wrapper(self, animal):
        if(isinstance(animal, Caine)):
            print("pisica " +self.num+ "fuge de cainele" +animal.num)
        else:
            print("interactiune incompatibila intre fiinte")
    return wrapper
```

```
class Mamifer:
    def __init__(self, nume):
        self.num= nume
```

```
    def spuneCeE(self):
        pass
```

```
class Om(Mamifer):
    def spuneCeE(self):
        print("Sunt un om si ma cheama:" +self.num)
```

```
    @interactiunea_om_pisica
    def interactioneaza(self, pisica):
        pass
```

```
class Femeie(Om):
    def spuneCeE(self):
        super().spuneCeE(self)
        print("si sunt femeie")
```

```
class Animal(Mamifer):
    def spuneCeE(self):
        print("Sunt animal si ma chema: " +self.num)
```

```
class Caine(Animal):
    def spuneCeE(self):
        super().spuneCeE(self)
        print("Si sunt caine")
```

```
class Pisica(Animal):
    def spuneCeE(self):
```

```

    super().spuneCeE(self)
    print("Si sunt pisica")

    @interactiune_pisica_caine
    def interactioneaza(self, animal):
        pass

class BridgeAbstract:
    def __init__(self, tip:Mamifer):
        self.tip=tip

    def spuneCeva(self):
        self.tip.spuneCeE()

class BridgeSpecializat(BridgeAbstract):
    def spuneCeva_concret(self):
        self.spuneCeva()
        print("Concret")

if __name__ == '__main__':
    pisi = Pisica("Pisi")
    cutu = Caine("Cutu")
    Ana = Om("Ana")
    pod = BridgeSpecializat(Ana)
    pod.spuneCeva_concret()
    Ana.interactioneaza(pisi)
    pisi.interactioneaza(cutu)

```

- **Kotlin:** Sa se creeze in Kotlin un functor (utilizat intr-o corutina) pentru o colectie de tip MutableMap care sa prelucreze valorile. Pentru testare, se vor lansa mai multe corutine pentru procesarea unui MutableMap in care cheile sunt valori intregi, iar valorile sunt string-uri care contin mai multe cuvinte separate prin spatiu. Apelurile functiei map din functor vor prelucra MutableMap-ul astfel incat: primul map va adauga prefixul “Test” la fiecare valoare iar al doilea map va apela functia toPascalCase

```

package com.pp.laborator
import kotlin.coroutines.*
import javax.xml.crypto.dsig.Transform

class MapFunctor<K, V>(private val map: MutableMap<K, V>){
    fun map(transform: (V) -> V){
        map.forEach{ (key, value) -> map[key] =transform(value)

```

```

    }
}

fun addPrefix(value: String):String {
    return "Test $value"
}

fun toPascalCase(value: String):String{
    return value.split(" ").joinToString(" "){ it.capitalize() }
}

fun main ()=runBlocking {
    val mutableMap: MutableMap<Int, String> = mutableMapOf(
        1 to "hellor world",
        2 to "kotlin is awesome",
        3 to "open chatgbt"
    )

    val functor = MapFunctor(mutableMap)

    //lansem 2 corutine pt a prelucra map-ul
    val job1 = launch {
        functor.map(::addPrefix)
    }

    val job2 = launch {
        functor.map(::toPascalCase)
    }

    //asteptam finalizarea corutinelor
    job1.join()
    job2.join()

    println(mutableMap)
}

```

PDF_mapReduce_IterTools

- **Python:** Pornind de la exemplu cu map_reduce din modulul more_itertools(python), sa se calculeze indexul invers pentru un set de document text(creat de student). Toate cele 3

functii (**key_func**, **value_func** si **reduce_func**) vor fi scrise ca expresii lambda. Se va citi continutul fiecarui document si apoi se va sparge si reuni intr-o singura lista de cuvinte
-> Functia de mapare va returna perechi de forma **<word, (document_id, 1)>**, in care cheia este cuvantul, iar valoarea este formata din tupla document_id(numele documentului), si 1(o aparitie a cuvantului in document)

-> Functia de reducere primeste toate perechile, emite o pereche de forma **<word, [(document_id_i, count_word_in_document_id_i), (...), ...]>**, unde al doile element din tupla reprezinta numarul de aparitii ale cuvantului in documentul respectiv. Pentru simplitate, functia de reducere poate utiliza functia **reduce_by_key** din biblioteca PyFunctional, ca sa creeze acea lista de tuple. Exemplu de output:

```
{ 'a':      [( 'text/1.txt',1), ( 'text/2.txt',2)],  
  'about':  [( 'text/3.txt',1)],  
  'always': [( 'text/3.txt',1)],  
  'are':    [(text/3.txt', 2)],  
  ....}
```

```
import more_itertools  
import functional  
from functional import seq  
  
if __name__ == '__main__':  
    continut1 = open("text1.txt").read().split(" ")  
    continut2 = open("text2.txt").read().split(" ")  
  
    keyFun = lambda cuv: cuv  
    valueFun = lambda cuv: ('data1.txt', 1)  
    valueFun2 = lambda cuv: ('data2.txt', 1)  
    reduceFun = lambda lista: seq(lista).reduce_by_key(lambda x, y: x + y)  
  
    rezultat1 = more_itertools.map_reduce(continut1, keyFun, valueFun, reduceFun)  
    rezultat2 = more_itertools.map_reduce(continut2, keyFun, valueFun2, reduceFun)  
  
    print(rezultat1)  
    print(rezultat2)  
  
    for key in rezultat1.keys():  
        if key in rezultat2.keys():  
            rezultat1[key] = [rezultat1[key].first(), rezultat2[key].first()]  
    print(rezultat1)
```

- **Python:** Dintr-un fisier de minim 50 de valori (numere) se iau cate 10 valori consecutive care vor fi procesate intr-un proces separat (din multiprocessing, Python) (deci se lanseaza in executie minim 5 procese). Datele sunt depuse intr-un ADT, apoi se va efectua o procesare de tip lambda care va gasi minimul, maximul si media din secventa, apoi va extrage o submultime ce contine numai numerele care se afla in intervalul media + sau - media patratica secventei procesate

```
import multiprocessing as mp
import math

#functie pentru a procesa o secventa de numere
def process_sequence(sequence):
    #calculam minimul, maximul si media
    min_val= min(sequence)
    max_val= max(sequence)
    mean_val= sum(sequence) / len(sequence)

    #calculam media patratica (radacina patrata a mediei patratelor)
    squared_mean = math.sqrt(sum(x**2 for x in sequence) /len(sequence))

    #definim intervalul
    lower_boound = mean_val - squared_mean
    upper_bound = mean_val + squared_mean

    #extragem numerele ce se afla in interval
    subset = [ x for x in sequence if lower_boound <= x <=upper_bound]

    return {
        'min': min_val,
        'max': max_val,
        'mean': mean_val,
        'subset': subset
    }

#functia principala
def main():
    #citirea numerelor din fisier
    with open('numbers.txt', 'r') as f:
        numbers= list(map(float, f.read().split()))

    #ne asiguram ca avem minim 50 de numere
    if len(numbers) < 50:
        raise ValueError("Fisierul trebuie sa contina minim 50 de numere")
```

```

#impartim numerele in secvente de cate 10
sequence = [ numbers [i:i + 10] for i in range(0, len(numbers), 10)]

#procesam fiecare secventa intr-un proces separat
with mp.Pool( processes=len(sequence)) as pool:
    results = pool.map(process_sequence, sequence)

#afisam rezultatele
for idx, result in enumerate(results):
    print(f"Secventa {idx + 1}")
    print(f" Min: {result['min']}")
    print(f" Max: {result['max']}")
    print(f" Mean: {result['mean']}")
    print(f" Subset: {result['subset']}\n")

if __name__ == '__main__':
    main()

```

- Folosind **with** ne asiguram ca fisierul va fi inchis automat dupa ce se termina blocul de cod
- sequences = [numbers[i:i + 10] for i in range(0, len(numbers), 10)] -> se face o secventa de indici ce merg de la 0 la lungimea listei cu un pas de 10, apoi pt fiecare indice i ia o sublista din numbers ce incepe cu indicele i si are 10 elemente
- with mp.Pool(processes=len(sequences)) as pool:
 - results = pool.map(process_sequence, sequences) -> se creeaza un **pool(grup de procese)** unde numarul de procese este egal cu numarul de secvente, astfel fiecare v fi procesata separat
 - pool.map(process_sequence, sequences) distribuie fiecare secventa catre functia "process_sequence" pe fiecare element din sequences
- ★ **f" "** este o metoda pentru a crea siruri de caractere formatare. Acestea simplifica inserarea si formatarea variabilelor sau a expresiilor in texte

- **Kotlin:** Utilizand fabrica de obiecte sa se scrie un program Kotlin care sa permita crearea unor obiecte de tip student differentiate de starea lui: integralist, restantier si repetent. La intrare programul va primi un fisier text cu inregistrarile a mai multor studenti(format nume & prenume, grupa, dictionar cu disciplina ca cheie si media finala ca valoare). Regulile de decizie pentru a afisa starea studentului sunt (nici o nota < 5 -integralist, numarul de cazuri cu nota < 5 este intre 2 si 4 -restantier, daca acest nr > 4 atunci studentul este clasificat ca repetent). Pentru cei restantieri si repetenti trebuie sa existe o metoda suplimentara de accesare bazata pe modelul Proxy a datelor dintr-o clasa de date cu verificarea unei parole. Se vor desena diagrame de clase si de obiecte

```
import java.io.File
```

```

abstract class Student( val nume: String, val Prenume: String, val medii: MutableMap<String, Double>)
{
    abstract fun souneCineEsti()
    open fun getDataConfifentiala(){
        println("accesul la date confidentiale este permis doar la restantieri si repetenti")
    }
}

```

```

class Integralist(val num:String, val Prenum: String, val med: MutableMap<String, Double>):
Student(num, Prenum, med)
{
    override fun souneCineEsti() {
        println("Ma cheama $num $Prenum, sunt integralist si am mediile: $med")
    }
}

```

```

class Restantier(val num: String, val Prenum: String, val med: MutableMap<String, Double>):
Student(num, Prenum, med)
{
    override fun souneCineEsti() {
        println("Ma cheama $num $Prenum, sunt restantier si am mediile: $med" )
    }

    override fun getDataConfifentiala() {
        val dataConfidentiala = ClasaDeDate()
        val proxy = ProtectieCuParola(dataConfidentiala)
        proxy.getDataConfidentiala(this.num)
    }
}

```

```

class Repetent(val num: String, val Prenum:String, val med: MutableMap<String, Double>):
Student(num, Prenum ,med)
{
    override fun souneCineEsti() {
        println("Ma cheama $num $Prenum, sunt REPETENT si am mediile : $med")
    }

    override fun getDataConfifentiala() {
        val dataConfidentiala = ClasaDeDate()
        val proxy= ProtectieCuParola(dataConfidentiala)
        proxy.getDataConfidentiala(this.num)
    }
}

```

```

}
class StudentFactory()
{
    fun makeStudent(tip: String, nume:String,prenume:String, medii:MutableMap<String, Double>):
Student
    {
        if(tip.lowercase() == "integralist"){
            return Integralist(nume, prenume, medii)
        }
        if(tip.lowercase() == "restantier"){
            return Restantier(nume, prenume, medii)
        }
        else
        {
            return Repetent(nume, prenume, medii)
        }
    }
}

fun citesteListaStudentiDinFisier( numef:String): List<Student>
{
    val SF = StudentFactory()
    val listaStudenti= mutableListOf<Student>()
    val studenti = File(numef).readLines()
    for (student in studenti)
    {
        val studentL = student.split(" ")
        val numeS = studentL[0]
        val prenumeS = studentL[1]
        val mediiS= studentL.filterIndexed { index, s -> index >= 2}

        val mediiPeMaterii = mediiS.windowed(2, 2)
        val HMPmed = mutableMapOf<String, Double>()
        for( element in mediiPeMaterii)
            HMPmed[element[0]] = element[1].toDouble()
        var contor = 0
        for( element in HMPmed)
            if(element.value < 5)
                contor++
        if (contor<1)
            listaStudenti.add(SF.makeStudent("integralist", numeS, prenumeS, HMPmed))
        else if (contor < 4)
            listaStudenti.add(SF.makeStudent("restantier", numeS, prenumeS, HMPmed))
        else
    }
}

```

```

        listaStudenti.add(SF.makeStudent("repetent", numeS, prenumeS, HMPmed))
    }
    return listaStudenti
}

interface LucratorCuDate
{
    fun getDataConfidentiala( user: String)
}

class ClasaDeDate: LucratorCuDate{
    override fun getDataConfidentiala(user: String) {
        print ("Aceasta este o data confidentiala. Accesul a fost permis")
    }
}

class ProtectieCuParola( val c: LucratorCuDate): LucratorCuDate
{
    val HMPparole = mapOf(Pair("Dorel", "doru123"), Pair("George", "geo123"))
    override fun getDataConfidentiala(user: String) {
        println(" Introduceceti parola pentru utilizatorul $user pentru a verifica daca aveti acces la datele
protejate: ")
        val parola = readln()
        if(HMPparole[user] == parola)
        {
            c.getDataConfidentiala(user)
        }
        else
        {
            println("Accesul a esuat, parola gresita")
        }
    }
}

fun main(args: Array<String>)
{
    val LStudenti = citesteListaStudentiDinFisier("catalog.txt")
    for (student in LStudenti)
        student.souneCineEsti()

    for (student in LStudenti)
        student.getDataConfifentiala()
}

```

PDF_procesePY_coroutineKT

- **Python:** Utilizand procese Python, sa se creeze un proces care da timestamp-ul la solicitarea unui alt proces care citeste cate un cuvant dintr-un fisier text. Vor exista minim 2 procese care citesc din mai multe fisiere (fiecare proces cate un fisier). Rezultatele sunt depuse intr-un alt ADT de tip dictionar, in care cheia va fi timestamp-ul, iar valoarea va fi cuvantul. Printr-o coada thread-safe, se va transmite ADT-ul creat catre un alt proces care il va prelucra astfel incat sa concateneze cheia si valoarea cu - intre ele, apoi sa separe cuvintele prin virgula si sa le scrie intr-un fisier (CSV).

```
import multiprocessing
import time
from multiprocessing import Queue, Lock, Process

def putInHashMap(coadaMesaje:Queue, coadaTimpi:Queue, CUV: str, HMP:dict):
    print("Se cere un timestamp")
    print(coadaMesaje.empty())
    timestamp = coadaTimpi.get()
    data = {timestamp: CUV} #se creeaza un dictionar temporar cu un singur elem, unde cheia e
    timestamp si valoarea e CUV
    HMP.update(data) #se adauga dictionarul temporar la HMP
    print("S-a facut adaugarea cuv in hMP")
    print(HMP)

def TimeStamp(coadaMesaje:Queue, coadaTimpi:Queue,noConsumers):
    consumate = 0 #un contor pt a uramari cate procese consumatoare (WordEater) au terminat
    print("a pornit procesul de timestamp")
    print("TimeStamp asteapta mesaj")
    while True: #se opreste cand toate procesele consumatoare sunt terminate
        mesaj = coadaMesaje.get()
        print("TimeStamp a primit mesajul: " + mesaj)
        if mesaj == "Timestamp":
            print("S-a trimis timestamp")
            coadaTimpi.put(time.time_ns()) #pune timestamp( timpul in nanosecunde)
        if mesaj == "Exit":
            consumate = consumate+1
            if(consumate==noConsumers): #daca toate procesele au terminat
                exit(0)

def WordEater(coadaMesaje:Queue, coadaTimpi:Queue, dictionar:dict, numeFis: str, returnValue:
```

Queue):

```
print("a pornit word eater")
continutFisier = open(numeFis, 'r').readline().split(" ")
print(continutFisier)
for cuv in continutFisier:
    print("se pune cuvantul: " + cuv + " in hashMap")
    coadaMesaje.put("Timestamp")
    print(coadaMesaje.empty()) #afiseaza daca coada este goala sau nu
    putInHashMap(coadaMesaje,coadaTimpi, cuv,dictionar) #apeleaza functia
    print("S-a reusit! ")
coadaMesaje.put("Exit")
return Value.put(dictionar)
exit(0)
```

def Concatenator(coadaInput:Queue):

```
dict1:dict = coadaInput.get() #preiau 2 dictionare din coada "coadaInput"
dict2:dict = coadaInput.get() #acestea 2 contin timestampuri si cuvinte colectate de cele 2 procese
```

WordEater

```
listaMeaFinala = [] #facem o lista goala ce va contine timestampurile si cuvintele combinate
for cheie in dict1:
    listaMeaFinala.append(str(cheie)) #adaugam timestamp-ul ca string in lista finala
    listaMeaFinala.append(dict1[cheie]) #adaugam cuvantul asociat timestampului
for cheie in dict2:
    listaMeaFinala.append(str(cheie))
    listaMeaFinala.append(dict2[cheie])
fisierOutput = open('fisierOut','w')
fisierOutput.write(str(listaMeaFinala).replace("[","," ).replace("\",","))
```

if __name__ == '__main__':

```
myLock = Lock() #previn accesul simultan la o resursa partajata intre mai multe procese
myHashMap = dict()
coadamsg: Queue = multiprocessing.Queue()
coadaReturn: Queue = multiprocessing.Queue()
coadaTimeStamp = multiprocessing.Queue()
```

```
timeStamper = Process(target=TimeStamp, args=(coadamsg,coadaTimeStamp,2))
```

#se va rula functia TimeStamp. Procesul primeste coada de mesaje, coada pt timestamp-uri si
numarul de cnsumatori 2

```
wordEater1 = Process(target=WordEater, args=(coadamsg, coadaTimeStamp,
myHashMap,"data1.txt",coadaReturn))
wordEater2 = Process(target=WordEater, args=(coadamsg, coadaTimeStamp,
```

```
myHashMap,"data2.txt",coadaReturn))
concatenate = Process(target=Concatenator, args=(coadaReturn,))
```

```
wordEater1.start()
wordEater2.start()
timeStamper.start()
concatenate.start()
```

```
timeStamper.join()
wordEater1.join()
wordEater2.join()
concatenate.join()
```

→ **Consumator** este un termen folosit pt a descrie un proces sau un fir de executie(thread) ce preia si proceseaza date dintr-o structura de date partajata, cum ar fi o coada

- **Kotlin:** Sa se scrie un program Kotlin care va utiliza mai multe corutine ce vor suma cate 2 valori alaturate intr-un hashmap, apoi rezultatele sumarilor vor fi depuse intr-un ADT (comun pt toate corutinele) rezultat. Inainte de afisare,se va utiliza o reimplementare a barierei care se va baza pe corutine si pe mecanisme de tip wait, notify si notify all.

```
package com.pp.laborator
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.async
import kotlinx.coroutines.runBlocking
import java.util.concurrent.ConcurrentHashMap
import kotlinx.coroutines.*
```

```
//HashMap initial
```

```
val hashMap = hashMapOf(
    1 to 2,
    2 to 4,
    3 to 6,
    4 to 8,
    5 to 10,
    6 to 12
)
```

```
//ADT comun pt a stoca rezultatele
```

```
val result = ConcurrentHashMap< Int, Int>()
```

```
//definirea unei bariere utilizand mecanisme de tip wait si notify
```

```
class CoroutineBarrier ( private val numberOfParties : Int){ //numberOfParties->nr ul de corutine ce trebuie sa ajunga la bariera
```

```

private var count = numberOfParties

@Synchronized
fun await() { //metoda sincronizata care scade controlul
    count-= 1
    if (count > 0){
        (this as java.lang.Object).wait() //daca mai sunt metode ce trebuie sa ajunga la bariera wait
        blocheaza firul de executie
    }
    else {
        (this as java.lang.Object).notifyAll() //daca toate corutinele au ajuns, notifyAll deblocheaza
        toate corutinele
    }
}
}

//implementarea corutinei si implementarea barierei
fun main()= runBlocking {
    val barrier = CoroutineBarrier(hashMap.size /2) //bariera se va activa cand jumătate din corutine
    vor ajunge la ea
    //lista job urilor (corutinele)
    val jobs = mutableListOf<Job>()

    for ( i in 1 until hashMap.size) {
        val firstValue = hashMap[i]
        val secondValue = hashMap[i + 1]

        //lansam corutine pentru a aduna valorile adiacente
        val job = launch(Dispatchers.Default){ //pt fiecare pereche de valori se lanseaza o corutina
            if( firstValue != null && secondValue != null) {
                val sum = firstValue + secondValue
                result[i] = sum

                println("Suma valorilor $firstValue si $secondValue = $sum")

                //Asteptam la bariera
                barrier.await() //awit este folosit pt a bloca corutina la bariera pana cand toate corutinele
                ajung in acest punct
            }
        }
        jobs.add(job)
    }
    //asteptam ca toate corutinele sa se termine
    jobs.forEach { it.join() } //metoda join() este apelata pt fiecare job(cortina) asigurandu-se ca

```

programul nu se incheie pana la finalizarea tuturor corutinelor

```
//afisam rezultatele finale
println("Rezultatele sumarilor: $result")
}
```

O bariera este un mecanism de sincronizare folosit in programele concurente sau paralele pentru a coordona executia mai multor fire de executie sau corutine

Ideea principala a unei bariere este ca un anumit numar de fire de executie sau corutine trebuie sa ajunga la un anumit punct de sincronizare (bariera) inainte ca oricare dintre ele sa poata continua executia.

Functionare:

Waiting: cand o corutina sau un fir de executie ajunge la bariera, acesta se blocheaza, adica asteapta pana cand toate celelalte corutine ajung in acelasi punct

Release: Dupa ce toate corutinele au ajuns la bariera, ele sunt deblocate si pot continua executia, ceea ce asigura ca nicio corutina nu va trece de la un anumit punct critic inainte ca toate sa fie pregatite

PDF_2

- **Python:** Cu ajutorul modelului memento sa se creeze un program Python care va permite aplicarea succesiva a unor lambda functii peste o colectie initializata cu date citite dintr-un fisier si apoi va restaura obiectele la starea anterioara in momentul in care acest lucru este cerut (la consola). Se vor aplica urmatoarele functii:

-> **f1(x)= x+1 (daca x par), altfel f1(x)=x**

-> **f2(x)= 3x*x - 2*x +1**

-> **f3(x)= x[i] + x[i+1]** , unde i este indexul x-ului primit ca parametru

Se va desena diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID

```
from functional import seq
```

```
class Originator:
```

```
    def __init__(self):
```

```
        f = open('data.txt', 'r')
```

```
        self.listaNumere= f.readline().split(",")
```

```
        self.listaNumere = [eval(i) for i in self.listaNumere]
```

```
class Memento:
```

```
    def __init__(self, lista):
```

```
        self.savedList= [ i for i in lista ] #salvam o copie a listei "lista" ce reprezinta starea curenta
```

```

def getState(self):
    return self.savedList

def f1(self):
    self.listaNumere = seq(self.listaNumere).map(lambda it: it+1)

def f2(self):
    self.listaNumere = seq(self.listaNumere).map(lambda it: 3*it*it - 2*it +1)

def f3(self, i):
    i = int(i)
    if 0 <= i < len(self.listaNumere) - 1:
        self.listaNumere[i] = self.listaNumere[i] + self.listaNumere[i + 1]
    else:
        print("Index invalid.")
def save(self):
    return self.Memento(self.listaNumere)

def restore(self, snapshot: Memento):
    self.listaNumere= snapshot.savedList

```

```

class CareTaker:
    def __init__(self):
        self.origin = Originator()
        self.history = list()

    def doSomething(self):
        while True:
            print("introduceti optiunea: 1 = f1; 2=f2; 3=f3, 0 = restore previous state")
            optiune = input()
            print(optiune)

            if (optiune == '1'):
                print("Ati ales sa executati functia f1")
                self.history.append(self.origin.save())
                self.origin.f1()
                print(self.origin.listaNumere)
            if (optiune == '2'):
                print("Ati ales sa executati functia f2")
                self.history.append(self.origin.save())
                self.origin.f2()
                print(self.origin.listaNumere)
            if (optiune == '3'):

```

```

        print("Ati ales sa executati functia f3")
        self.history.append(self.origin.save())
        print("introduceti indexul pt f3")
        i = input()
        self.origin.f3(i)
        print(self.origin.listaNumere)
    if (optiune == '0'):
        print("Ati ales sa executati restaurare() ->")
        self.origin.restore(self.history.pop())
        print(self.origin.listaNumere)

if __name__ == '__main__':
    CT = CareTaker()
    CT.doSomething()

```

- **Kotlin:** Utilizand functii de extensie sa se scrie in Kotlin un aplicative care aplica intr-o corutina o transformare lambda bazata pe regex care sa elimine dintr-un continut HTML tag-urile 'script' si 'style' (si continutul lor, bineinteles). Functia extensie poate fi aplicata unui String, iar aceasta poate filtra continutul HTML

```

package com.pp.laborator
import kotlinx.coroutines.*
import java.util.regex.Pattern //folosita pt a lucra cu corutine in Kotlin

fun String.removeScriptAndStyletags() : String {
    //regex pt a gasi si elimina tag-urile si continutul <script> si <style>
    val scriptRegex="<script[^>]*>.*?</script>".toRegex(RegexOption.DOT_MATCHES_ALL)
    val styleRegex= "<style[^>]*>.*?</style>".toRegex(RegexOption.DOT_MATCHES_ALL)

    return this.replace(scriptRegex, "").replace(styleRegex, "")
}

fun main() = runBlocking {
    //exemplu continut HTML
    val htmlContent = """
        <html>
        <head>
        <style>
            body { background-color: #f3f3f3; }
        </style>
        <script>

```

```

        console.log("Hello, World!");
    </script>
</head>
<body>
    <h1>Title</h1>
    <p>This is a paragraph.</p>
</body>
</html>
"".trimIndent()

//lansam o corutina pentru a filtra continutul html
val filteredContent = withContext(Dispatchers.Default){
    htmlContent.removeScriptAndStyletags()
}

//afisare continut
println(filteredContent)
}

```

SV 32

- **Kotlin:** Utilizand Kotlin sa se splice un functor peste elementele unui hashmap care va realiza urmatoarele procesari: va aplica $f(x) = 3x-1$ ca lambda, le va transforma in string si va afisa rezultatul

```

class HashMapFuncutor< T, K>(val hashMap: HashMap<T, K>)
{
    fun map(function: (K) ->(K)): HashMapFuncutor<T, K>
    {
        var result = HashMap<T, K>()
        for((key, value) in hashMap){
            result[key] = function(value)
        }
        return HashMapFuncutor(result)
    }
}

fun main(args: Array<String>) {

    val mapa= hashMapOf<Int, Int>(Pair(1,1), Pair(2,2), Pair(3,3))
    println(HashMapFuncutor(mapa).map { elem -> 3*elem -1 }.hashMap.toString())
}

```

SV 32

- **Python:** Utilizand functii din itertools -Python si un contor tip closure sa se genereze automat nume de fisiere temporare care vor fi create astfel s1-index-s2.tmp unde s1 si s2 sunt nume primite la apelul metodei. Metoda va verifica unde a ramas generatorul si va intoarce urmatorul nume

```
import itertools

def file_name_generator(s1, s2):
    #cream un generator care va genera numere incepand de la 1
    counter = itertools.count(1)

    #closure care va genera numele de fisier
    def generate_name():
        index = next(counter) #obtinem urmatorul index
        return f"{s1}-{index}-{s2}.tmp"
    return generate_name

s1 = "temp"
s2 = "data"

generate_temp_file_name = file_name_generator(s1, s2)

#generam cateva nume de fisiere
print(generate_temp_file_name())
print(generate_temp_file_name())
print(generate_temp_file_name())
```

SV 33

- **Kotlin:** Utilizand Kotlin functiile pentru procesare submultimi din se va prelua un fisier text(creat de programator cu cateva propozitii in el) si se vor extrage 2 caractere din mijlocul cuvantului daca cuvantul are minim 4 caractere. Se va utiliza combinatie cu lambda peste colectii pentru procesare

```
import java.io.File
```

```

fun main(args: Array<String>)
{
    var linie = File("incercare.txt").readText().split(" ")
    println(linie)

    var lista = linie.asSequence().filter { word -> word.length > 4 }
        .map { word -> word.substring(startIndex = word.length/2, endIndex= word.length/2+2)
    }.toList()

    println(lista)
}

```

→ **asSequence()**: converteste o colectie(o lista de cuvinte in acest caz) intr-o secventa Sequence
 → **Substring**: este folosit pt a extrage un substring din mijlocul fiecarui cuvnt

- **Python:** Utilizand modelul mediator sa se creeze un program Python care sa permita comunicarea prin mesaje intre niste obiecte de tip furnica (e.g mesaje, mancare, drum bun, pericol, ajutor). Se vor desena diagrama de clase si obiecte. Se va explica maniera de aplicare a principiilor SOLID

```

from abc import ABC, abstractmethod

#interfata mediator
class Mediator(ABC):
    @abstractmethod
    def trimite_mesaj(self, mesaj, furnica):
        pass

#implementare mediator
class ChatMusuroi(Mediator):
    def __init__(self):
        self.furnici = []

    def adauga_furnica(self, furnica):
        self.furnici.append(furnica)

    def trimite_mesaj(self, mesaj, furnica):
        for f in self.furnici:
            if f != furnica:
                f.printeaza_mesaj(mesaj, furnica)

```

```

#clasa de baza pt furnici
class Furnica:
    def __init__(self, nume, mediator):
        self.nume= nume
        self.mediator= mediator
        self.mediator.adauga_furnica(self)

    def spune(self, mesaj):
        self.mediator.trimite_mesaj(mesaj, self)

    def printeaza_mesaj(self, mesaj, expeditor):
        print(f"[{expeditor.nume} zile]: {mesaj}")

    def __str__(self):
        return self.nume
#exemplu utilizare
if __name__ == '__main__':
    chat = ChatMusuroi()

    furnicaMica= Furnica("Furnica Mica", chat)
    furnicuta= Furnica("Furnicuta", chat)
    furnicul= Furnica("Furnicul", chat)

    furnicaMica.spune("Drum bun!")
    furnicuta.spune("mancare")
    furnicul.spune("Pericol")

```

SV 39

- **Kotlin:** Fie o multime la intrare A de 100 de elemente pare alese aleator care sunt depuse intr-o colectie. Sa se scrie un program in care se va calcula $b_n = a_1^2 + a_2^2 + a_3^2 + \dots + a_n^2$, $\forall$ n natural, diferit de 0.

```

import kotlin.random.Random

fun main(args: Array<String>) {

    //generam o lista de 100 de numere pare aleatorii
    val A = List(100) {Random.nextInt(1, 1000)*1}

    //calculam suma patratelor fiecarui elemeny din lista A

```

```

val bn= A.sumOf { it * it}

//Afisam rezultatul
println("Suma patratekor elementelor din lumtimea A este : $bn")
}

```

- **Pthon:** utilizand tkinter sa se creeze un program Python care permite introducerea datelor unui angajat in companie (companie, departament, nume, data de nastere, functie si salariu). Pentru introducerea datei de nastere se va folosi un combobox pentru selectia lunii. Programul va avea meniu.

```

import tkinter as tk
from tkinter import ttk
from tkinter import messagebox

class EmployeeForm(tk.Tk): #mosteneste Tk din tkinter => EmployeeForm este o fereastră Tk
    def __init__(self):
        super().__init__() #apelam constructorul clasei tk.Tk pt a initializa corect fereastra

        self.title("Employee Data Entry")
        self.geometry("400x300")

        # Meniu
        menu = tk.Menu(self) #se creeaza un obiect "Menu" ce se asociaza derestrei principale self
        self.config(menu=menu) #configureaza fereastra principala sa foloseasca acest meniu

        file_menu = tk.Menu(menu, tearoff=0) #creeaza un submeniu numit "file_menu" si seteaza
        tearoff=0, ceea ce inseamna ca
        #utilizatorul nu poate separa meniul de fereastră
        menu.add_cascade(label="File", menu=file_menu) #adauga acest submeniu in meniul principal
        cu eticheta File
        file_menu.add_command(label="Exit", command=self.quit) #adauga o comanda in submeniul
        File

        help_menu = tk.Menu(menu, tearoff=0) #Se creeaza un alt submeniu numit "help_menu"
        menu.add_cascade(label="Help", menu=help_menu) #se adauga submeniul in meniul principal
        help_menu.add_command(label="About", command=self.show_about) #adauga o comanda pt
        submeniul help

        # Labels and Entries

```

```

tk.Label(self, text="Company:").grid(row=0, column=0, padx=10, pady=5, sticky="w")
#creeaza o eticheta(label) cee afis textul
#padx=adauga un ps de pixeli pe oriz si 5 pe verticala si aliniaza textul la stg ( W->west)
self.company_entry = tk.Entry(self) #creeaza un camp de introducere Entru pentru a permite
utilizatorului sa introduca numele
self.company_entry.grid(row=0, column=1, padx=10, pady=5) #plaseaza acest camp de
introducere in randul 0, coloana 1

tk.Label(self, text="Department:").grid(row=1, column=0, padx=10, pady=5, sticky="w")
self.department_entry = tk.Entry(self)
self.department_entry.grid(row=1, column=1, padx=10, pady=5)

tk.Label(self, text="Name:").grid(row=2, column=0, padx=10, pady=5, sticky="w")
self.name_entry = tk.Entry(self)
self.name_entry.grid(row=2, column=1, padx=10, pady=5)

tk.Label(self, text="Date of Birth:").grid(row=3, column=0, padx=10, pady=5, sticky="w")
self.day_entry = tk.Entry(self, width=5) #latimea campului va fi de 5 caractere
self.day_entry.grid(row=3, column=1, padx=10, pady=5, sticky="w")

self.month_combobox = ttk.Combobox(self, values=[ #creeaza un combobox
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"])
self.month_combobox.grid(row=3, column=1, padx=10, pady=5)
self.month_combobox.current(0) #seteaza luna implicita

self.year_entry = tk.Entry(self, width=5)
self.year_entry.grid(row=3, column=2, padx=10, pady=5)

tk.Label(self, text="Position:").grid(row=4, column=0, padx=10, pady=5, sticky="w")
self.position_entry = tk.Entry(self)
self.position_entry.grid(row=4, column=1, padx=10, pady=5)

tk.Label(self, text="Salary:").grid(row=5, column=0, padx=10, pady=5, sticky="w")
self.salary_entry = tk.Entry(self)
self.salary_entry.grid(row=5, column=1, padx=10, pady=5)

# Save button
save_button = tk.Button(self, text="Save", command=self.save_data) #creeaza un buton save.
cand utilizatorul apasa butonul, metoda save_data va fi apelata
save_button.grid(row=6, column=0, columnspan=2, pady=10)

def save_data(self): #obține valoarea introdusa de utilizator in campurile date
    company = self.company_entry.get()

```

```

department = self.department_entry.get()
name = self.name_entry.get()
day = self.day_entry.get()
month = self.month_combobox.get()
year = self.year_entry.get()
position = self.position_entry.get()
salary = self.salary_entry.get()

# Optionally, you can add code here to save the data to a file or database.
messagebox.showinfo("Success", f"Data for {name} has been saved!")

def show_about(self):
    messagebox.showinfo("About", "Employee Data Entry\nVersion 1.0")

if __name__ == "__main__":
    app = EmployeeForm()
    app.mainloop()

```

SV 46

- **Kotlin:** Pornind de la doua multimi A si B care contin 20 de elemente depuse in doua colectii separate si tinand cont de $A \times B = \{(a,b) \mid a \in A \vee b \in B\}$, sa se scrie un program Kotlin care va calcula $(A \times B) \cap (B \times A)$ utilizand functii specifice colectiilor si eventual lambda calculul urmatoarele. Rezultatul este depus intr-un dictionar iar acesta va fi afisat

```

fun main() {
    // Generăm două mulțimi A și B de 20 de elemente fiecare
    val A = List(20) { it + 1 } // Mulțimea A = [1, 2, 3, ..., 20]
    val B = List(20) { it + 10 } // Mulțimea B = [10, 11, 12, ..., 29]

    // Calculăm produsul cartezian A x B
    val AxB = A.flatMap { a -> B.map { b -> a to b } }

    // Calculăm produsul cartezian B x A
    val BxA = B.flatMap { b -> A.map { a -> b to a } }

    // Calculăm intersecția dintre AxB și BxA
    val intersection = AxB.intersect(BxA.toSet())
    println(intersection)
}

```

```
// Depunem rezultatul într-un dicționar
val result = intersection.groupBy({ it.first }, { it.second })

// Afișăm dicționarul
println("Intersecția dintre AxB și BxA este:")
result.forEach { (key, value) -> println("$key, $value") }

result.toMap()
println(result)
}
```

- **Python:** Utilizand modelul strategie sa se creeze un program Python care sa trateze diferentiat un mesaj de eroare in functie de clasa lui (2- warning-> scriu in fisier de warnings, 1-error-> afisez la consola si scriu in fisier de erori comune, 0-critical error-> voi scrie in fisierul de erori grave si voi opri programul.) Se vor utiliza exceptii personalizate crate de programator. Se vor desena diagrama de clase si obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```
class Strategy:

    def trateazaEroare(self, mesaj: str):
        pass

class WarningStrategy(Strategy):
    def trateazaEroare(self, mesaj: str):
        f = open("warnings.txt", "a")
        f.write(mesaj)

class ErrorStrategy(Strategy):
    def trateazaEroare(self, mesaj: str):
        f=open("errors.txt", "a")
        f.write(mesaj)
        print(mesaj)

class fatalErrorStrategy(Strategy):
    def trateazaEroare(self, mesaj: str):
        f= open("fatalErrors.txt", "a")
        f.write(mesaj)
```

```

    print(mesaj)
    #exit(0)

class Context:
    def __init__(self):
        self.strtg = Strategy()

    def setStrategy(self, strategie: Strategy):
        self.strtg = strategie

    def execute(self, mesaj):
        self.strtg.trateazaEroare(mesaj)

class ContextEstablisher:
    def __init__(self):
        self.context = Context()

    def EstablishErrorType(self, mesaj:str):
        if(mesaj.startswith("2")):
            self.context.setStrategy(WarningStrategy())
        else:
            if(mesaj.startswith("1")):
                self.context.setStrategy(ErrorStrategy())
            else:
                if(mesaj.startswith("0")):
                    self.context.setStrategy(fatalErrorStrategy())
                else:
                    print("Mesaj de eroare invalid")

    def Processerros(self, mesaj:str):
        self.EstablishErrorType(mesaj)
        self.context.execute(mesaj)

if __name__ == "__main__":
    mesajWarning = "2 - warning! va fi scris doar in fisier"
    mesajEroare = "1 - eroare! va fi scrisa in fisier si in consola"
    mesajEroareFatala = "0 - eroare fatala! va fi scrisa in fisier si in consola"
    CE = ContextEstablisher()
    CE.Processerros(mesajEroareFatala)
    CE.Processerros(mesajEroare)
    CE.Processerros(mesajWarning)

```

- **Python:** Cu ajutorul modelului memento sa se creeze un program Python care va permite aplicarea succesiva a unor functii lambda peste o colectie si apoi va restura obiectele la starea anterioara in momentul in care acest lucru este cerut la consola

$f1(x) = x+1$ (daca x par), altfel $f1(x)=x$

$f2(x)=3x*x -2*x+1$

$f3(x,y)=x+y$

Se vor desena diagrama de clase si obiecte. Se va aplica maniera de aplicare a principiilor SOLID

```
class Memento:
    def __init__(self, lista):
        self.listaNumere= lista

    def get_state(self):
        return self.listaNumere

class Originator:
    def __init__(self):
        f = open('data.txt', 'r')
        self.listaNumere = list(map(int, f.readline().split(",")))

    def set_state(self, lista):
        self.listaNumere= lista

    def get_state(self):
        return self.listaNumere

    def f1(self):
        """Aplică funcția lambda f1: x+1 (dacă x este par), altfel x."""
        functie = lambda x: x + 1 if x % 2 == 0 else x
        self.listaNumere = [functie(x) for x in self.listaNumere]

    def f2(self):
        """Aplicam functia f2"""
        functie = lambda x: 3 * int(x) * int(x) - 2 * int(x) + 1
        self.listaNumere= [ functie(x) for x in self.listaNumere]

    def f3(self, y):
        """Aplicam functia f3"""
        y=int(y)
        functie= lambda x: int(x) +y
```

```

self.listaNumere = [functie(x) for x in self.listaNumere]

def create_memento(self):
    return Memento(self.listaNumere.copy())

def restore(self, memento):
    self.listaNumere = memento.get_state()

class Caretaker:
    def __init__(self):
        self.originator= Originator()
        self.history = list()

    def doSomething(self):
        while True:
            print("introduceti optiunea: 1 = f1; 2= f2; 3= f3, 0= restore previous state")
            optiune = input()
            print(optiune)

            if (optiune == '1'):
                print("Ati ales sa aplicati functia 1")
                self.history.append((self.originator.create_memento()))
                #apelurile create_memento() salveaza starea curenta a obiectului Originator in self.history
                self.originator.f1()
                print(self.originator.listaNumere)

            if (optiune == '2'):
                print("Ati ales sa aplicati functia 2")
                self.history.append((self.originator.create_memento()))
                self.originator.f2()
                print(self.originator.listaNumere)

            if (optiune == '3'):
                print("Ati ales sa aplicati functia 3")

                self.history.append((self.originator.create_memento()))
                print("introduceti y")
                y= input()
                self.originator.f3(y)
                print(self.originator.listaNumere)

            if (optiune == '0'):
                print("Ati ales sa aplicati functia 0")
                self.originator.restore(self.history.pop())

```

```

        print(self.originator.listaNumere)

if __name__ == '__main__':
    CT= Caretaker()
    try:
        CT.doSomething()
    except KeyboardInterrupt:
        print("\nProgram intrerupt de utilizator")

```

SV 53

- **Kotlin:** Fie doua colectii cu 100 de numere din submultimile

$$A = \{ z \in N \mid x = \frac{8n-18}{2n-9}, n \in N \} \text{ si } B = \{ z \in N \mid x = \frac{9n^2-48n+18}{3n-8}, n \in N \} .$$

Pentru acestea se vor calcula (crea program in Kotlin) utilizand functii de transformare specifice si lambda calcul urmatoarele operatii $(A \times B) \cup (B \cap A)$ unde $A \times B = \{(a,b) \mid a \in A \wedge b \in B\}$. Rezultatul este depus intr-un hashmap si acesta va fi afisat

```

//functia pentru generarea colectiei A
fun generateCollectionA(size: Int): Set<Int> {
    val collection = mutableSetOf<Int>()
    var n = 1
    while (collection.size < size) {
        val numerator = 8 * n - 182
        val denominator = n - 9
        if (denominator != 0) {
            val value = numerator / denominator
            collection.add(value)
        }
        n++
    }
    return collection
}

fun generateCollectionB(size: Int): Set<Int> {
    val collection = mutableSetOf<Int>()
    var n = 1
    while (collection.size < size) {
        val numerator = 9 * n * n - 48 * n + 18
        val denominator = 3 * n - 8
        if (denominator != 0) {
            val value = numerator / denominator

```

```

        collection.add(value)
    }
    n++
}
return collection
}

//functie pt produs cartezian
fun cartezian( A: Set<Int>, B: Set<Int>): Set<Pair<Int, Int>>{
    return A.flatMap { a -> B.map { b -> Pair(a, b)} }.toSet()
}

//functie pentru intersectie
fun intersectie( B:Set<Int>, A:Set<Int>): Set<Int>{
    return B.intersect(A)
}

fun main(){
    val size = 100
    val A= generateCollectionA(size)
    val B= generateCollectionB(size)

    val axb= cartezian(A, B)

    val bIntersectA = intersectie(B, A)

    val finalresult = hashSetOf<Any>().apply{
        addAll(axb)
        addAll(bIntersectA)
    }

    println("rezultatul este:")
    println(finalresult)
}

```

- **Python:** Sa se scrie un program Python (utilizand threading) care proceseaza simultan utilizand mai multe thread-uri un hashmap X si un dictionar Y cu formula $f(x, y) = (x * y_i + y_{i+1})$ iar rezultatul este depus in yi. Dictionarul are pereche de valoare si index i. Se va utiliza rlock()

```

import threading

#initializare RLock()
lock = threading.RLock()

#functis de procesare a formulei f(x, y)
def formula(X, Y, i):
    with lock:
        if i < len(Y) - 1:
            Y[i]= X[i] * Y[i] + Y[i+1]
        else:
            Y[i] = X[i] + Y[i] #pt ultimul elemn nu avem Y[i+1]

#ffunctie pt thread uri
def run_threads(X, Y):
    threads = []
    for i in range(len(Y)):
        thread= threading.Thread(target= formula, args=(X, Y, i))
        threads.append(thread)
        thread.start()

    #asteptam ca toate thread-urile sa isi termine executia
    for thread in threads:
        thread.join()

#ex hashmap X si dictionar y
X = { 0: 1, 1: 2, 2: 3, 3: 4}
Y = {0: 10, 1: 20, 2: 30, 3: 40}

#rulare multithread
run_threads(X, Y)

print(Y)

```

SV 76

- **Kotlin:** Utilizand Kotlin si OOP sa se creeze un program care permite modelarea unei sali de laborator cu tot cu operatii pentru obiecte. Se vor folosi liste mutabile. Se va desena diagrama UML(clase si obiect). Se va explica maniera de aplicare a principiilor SOLID

```

class Computer(val id: String, val specs: String)

```

```

class Equipment(val name: String, val type: String)

class User(val name: String, val role: String)

class Lab(val name: String, val capacity: Int){
    private val computers= mutableListOf<Computer>()
    private val equipments = mutableListOf<Equipment>()
    private val users = mutableListOf<User>()

    //adaugarea unui calculator in laborator
    fun addComputer(computer: Computer)
    {
        if (computers.size < capacity){
            computers.add(computer)
        }else {
            println("Capacitatea laboratorului a fost atinsa")
        }
    }
    fun addEquipment( equipment: Equipment){
        equipments.add(equipment)
    }
    fun adduser(user: User){
        users.add(user)
    }

    fun displayInfo(){
        println(" Lab: $name")
        println("Capacity: $capacity")
        println("Computers:")
        computers.forEach{ println(" - ${it.id}: ${it.specs}")}
        println("Equipmrnts:")
        equipments.forEach{ println( " - ${it.name} (${it.type})")}
        println("Users:")
        users.forEach{println(" - ${it.name} (${it.role})")}
    }
}

fun main(){
    val lab = Lab( "Lab 1", 5)

    val computer1= Computer("PC1", "Intel7, 16GB RAM")
    val computer2= Computer("PC2", "AMD ryzen, 32GB RAM")
    val equipment = Equipment(" Projector", "Optoma")
    val user1= User("John Dow", "Student")

```

```

val user2 = User("Dr. Smith", "Professor")

lab.addComputer(computer1)
lab.addComputer(computer2)
lab.addEquipment(equipment)
lab.adduser(user1)
lab.adduser(user2)

lab.displayInfo()

}

```

- **Python:** utilizand modelul pod pot sa se scrue un program Python care va desena in mod grafic un triunghi, un cerc si un dreptunghi. Se vor desena diagrame de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID

```

import tkinter as tk
from abc import ABC, abstractmethod

#Clasa abstracta forma
class Forma(ABC):
    @abstractmethod
    def deseneaza (self, canvas):
        pass

#Clasa pentru triunghi
class Triunghi(Forma):
    def __init__ (self, x1, y1, x2, y2, x3, y3):
        self.x1, self.y1= x1, y1
        self.x2, self.y2 = x2, y2
        self.x3, self.y3 = x3, y3

    def deseneaza(self, canvas):
        canvas.create_polygon([self.x1, self.y1, self.x2, self.y2, self.x3, self.y3], outline= 'black')

#Clasa pt cerc
class Cerc(Forma):
    def __init__ (self, x, y, r):
        self.x, self.y, self.r = x, y, r

```

```

def deseneaza(self, canvas):
    canvas.create_oval(self.x - self.r, self.y - self.r, self.x + self.r, self.y + self.r, outline='black',
fill='black')

class Dreptunghi(Forma):
    def __init__(self,x1,y1,x2,y2):
        self.x1, self.y1 = x1, y1
        self.x2, self.y2= x2, y2

    def deseneaza(self, canvas):
        canvas.create_rectangle(self.x1, self.y1, self.x2, self.y2, outline='black', fill='black')

def main():
    root = tk.Tk()
    root.title("Desenare Forme")
    canvas = tk.Canvas(root, width=400, height=400)
    canvas.pack()

    forme = [
        Triunghi(100, 150, 150, 50, 200, 150),
        Cerc(300, 200, 50),
        Dreptunghi(50, 250, 150, 350)
    ]

    #desenare forme
    for forma in forme:
        forma.deseneaza(canvas)

    root.mainloop()

if __name__ == '__main__':
    main()

```

SV 81

- **Kotlin:** Utilizand modelul adaptor sa se creeze un program Kotlin care va primi la intrare diverse tipuri de date simple sau de tip colectie si va realiza scrierea lor intr-un

fișier ca o singură operație care primește doar obiectul și nume fișier. Se vor desena diagrama de clase și de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```
import kotlin.js.ExperimentalJsFileName

//interfața Target care definește metoda comună
interface Target {
    fun write( data: Any, fileName: String)
}

//Concrete adapter care implementează target și definește metoda write
class FileWriterAdapter: Target {
    override fun write( data: Any, fileName: String){
        val file= java.io.File(fileName)
        file.bufferedWriter().use { out ->
            when(data){
                is String -> out.write(data)
                is Int -> out.write(data.toString())
                is Double -> out.write(data.toString())
                is List<*> -> data.forEach { out.write(it.toString() + "\n")}
                is Map<*,*> -> data.forEach { (key, value) -> out.write("$key=$value\n") }
                else -> out.write("Unsuported data type")
            }
        }
    }
}

class Client (private val adapter: Target) {
    fun process(data: Any, fileName: String) {
        adapter.write(data, fileName)
    }
}

fun main(){
    val adapter = FileWriterAdapter()
    val client = Client(adapter)

    client.process("Hello world", "output.txt")
    client.process(1234, "output_int.txt")
    client.process(listOf("apple", "banana", "cherry"), "output_list.txt")
    client.process(mapOf("key1" to "value1", "key2" to 42), "output_msp.txt")
}
```

- **val file = java.io.File(filename)** -> se creeaza un obiecte File din pachetul java.io, utilizand calea sau numele fisierului dat ca argument (filename)
- **val file = java.io.File(filename)** -> creeaza un BufferedWriter, un obiect care permite scrierea de text in fisier intr-un mod eficient, utilizand un buffer intern. Acest lucru reduce numarul de accesari fizice ale discului, ceea ce face operatiunea de scriere mai rapida
 - **use** este o functie de extensie in Koylin care deschide resursa (BufferedWriter in acest caz) si se asigura ca aceasta este inchisa corect dupa ce blocul de cod s-a executat)
 - **out**-> acest simbol indica inceputul unui lambda, unde out este parametrul ce reprezinta obiectul BufferedWritre

- **Python:** Utilizand modelul comanda sa se scrie in Python un program care pornind de la o clasa student va asigura posibilitatea unor comenzi specifice unui obiect colega care sa conduca la schimbarea starii interne a unui obiect student (de ex din fericit in disperat). Se vor desena diagrama da clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```
from abc import ABC, abstractmethod

#clasa abstracta comanda
class Comanda(ABC):
    @abstractmethod
    def executa(self):
        pass

class Student:
    def __init__(self, nume):
        self.nume = nume
        self.stare= "Fericit"

    def schimba_starea(self, stare_noua):
        self.stare= stare_noua
        print(f" {self.nume} este acum {self.stare}")

#comenzi concrete
class ComandaFaceGlume(Comanda):
    def __init__(self, student):
        self.student = student

    def executa(self):
        self.student.schimba_starea("fericit")
```

```

class ComandaCritica(Comanda):
    def __init__(self, student):
        self.student= student

    def executa(self):
        self.student.schimba_starea("Trist")

class ComandaJigneste(Comanda):
    def __init__(self, student):
        self.student= student

    def executa(self):
        self.student.schimba_starea("Disperat")

class Colega:
    def trimite_comanda(self, comanda):
        comanda.executa()

student = Student("ion")
colega = Colega()

comanda_glume = ComandaFaceGlume(student)
comanda_critica = ComandaCritica(student)
comanda_jigneste= ComandaJigneste(student)

colega.trimite_comanda(comanda_glume)
colega.trimite_comanda(comanda_critica)
colega.trimite_comanda(comanda_jigneste)

```

SV 86

- **Kotlin:** Utilizand modelul observtor sa se creeze in pentru un obiect de tip recalculare a pretului, in functie de niste rate de reducere introduse extern de la tastatura (prin intermediul unui model proxy cu validare de user, parola), un logger (scriere automata in jurnal a operatiilor), care va scrie intr-un fisier separat user si data la care a fost efectuata modificarea de pret si ce modificare a fost realizata) Se vor desena diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```

import java.io.File
import java.text.SimpleDateFormat

```

```

import java.util.Date

interface Observer {
    fun update(pret: Double)
}

class Pret( private var valoare: Double){
    fun modificarePret(rateDeReducere: Double){
        valoare -= valoare * rateDeReducere/100
    }

    fun getPret(): Double {
        return valoare
    }
}

//recalculare pret
class RecalcularePret(private val pret: Pret){
    private val observers: MutableList<Observer> = mutableListOf()

    fun attach(observer : Observer){
        observers.add(observer)
    }
    fun detach(observer: Observer){
        observers.remove(observer)
    }

    fun notifyObservers() {
        for ( observer in observers) {
            observer.update(pret.getPret())
        }
    }

    fun recalcularePret( rateDeReducere: Double){
        pret.modificarePret(rateDeReducere)
        notifyObservers()
    }
}

//clasa LOGGER (observer)

class Logger( private val user : String): Observer {
    private val fisierLog = "lok.txt"

```

```

override fun update(pret: Double) {
    val data = SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(Date())
    scrieInLoG(user, data, "Pretul a fost modificat la $pret")
}

private fun scrieInLoG(user: String, data: String, modificare: String) {
    File(fisierLog).appendText("[$data] $user: $modificare\n")
}
}

//user proxy
class UserProxy( private val realSubject: RecalcularePret, private val user: String, private val parola:
String){
    private val userDb = mapOf("admin" to "admin123")

    private fun validate(): Boolean {
        return userDb[user] == parola
    }

    fun recalculeazaPret( rateDeReducere: Double) {
        if( validate()){
            realSubject.recalcularePret(rateDeReducere)
        }
        else{
            println("Autentificare esuata! acces refuzat!")
        }
    }
}

//clasa rate de reducere
class RateDereducere {
    fun introduceRate(): Double {
        println("Introduceti rata de reducere (%): ")
        return readLine()?.toDoubleOrNull() ?: 0.0
    }
}

fun main(){
    val pretInitial = 100.0
    val pret = Pret(pretInitial)
    val recalcularePret = RecalcularePret(pret)

    println(" introduceti user: ")

```

```

val user = readLine() ?: ""
println("introduceti parola: ")
val parola = readLine() ?: ""

val logger = Logger(user)
recalcularePret.attach(logger)

val proxy = UserProxy(recalcularePret, user, parola)
val rateDeReducere = RateDereducere()
val rate = rateDeReducere.introduceRate()

proxy.recalculeazaPret(rate)

}

```

- **Python:** Utilizand fabrica de fabrici sa se creeze un program Python care in functie de limba selectata in apel sa intoarca un buton a carui eticheta sa fie in limba selectata la apel si apoi sa fie afisat pe ecran utilizand tkinter. Se vor desena diagrama de clase si de obiecte. Se va explica maniera SOLID.

```

import tkinter as tk
from abc import ABC, abstractmethod

# Interfața pentru fabrica de butoane
class ButtonFactory(ABC):
    @abstractmethod
    def create_button(self, root):
        pass

# Fabrica de butoane pentru limba română
class RomanianButtonFactory(ButtonFactory):
    def create_button(self, root):
        return tk.Button(root, text="Apasă-mă")

# Fabrica de butoane pentru limba engleză
class EnglishButtonFactory(ButtonFactory):
    def create_button(self, root):
        return tk.Button(root, text="Click Me")

# Fabrica care decide ce buton să creeze pe baza limbii
class LanguageButtonFactory:

```

```

def get_factory(self, language_code):
    if language_code.lower() == 'romanian':
        return RomanianButtonFactory()
    elif language_code.lower() == 'english':
        return EnglishButtonFactory()
    else:
        raise ValueError(f"Limba '{language_code}' nu este suportată.")

def create_and_display_button(language_code):
    root = tk.Tk()
    root.geometry("300x100")

    factory = LanguageButtonFactory().get_factory(language_code)
    button = factory.create_button(root)
    button.pack(pady=20)

    root.mainloop()

if __name__ == "__main__":
    create_and_display_button('romanian')
    create_and_display_button('english')

```

SV 88

- **Kotlin:** Sa se creeze un program Kotlin care va utiliza subclase active (cu mutex si lock) pentru procesarea simultana a unui hasmap bazata. Clsa de baza va stabili operatiile (de ex adunare, scadere, inmultire sau impartire) iar subclasele vor pune la dispozitie obiecte care se executa in fire separate. Se va prezenta diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```

package com.pp.laborator

import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.*
import kotlinx.coroutines.sync.withLock

sealed class HashMapOps( public var hashmap: HashMap< Int, Int>) {
    fun sum(): Int{
        var sum: Int = 0
        hashmap.forEach{
            sum +=it.value
        }
    }
}

```

```

        return sum
    }

    fun mul(): Int{
        var mul: Int=1
        hashmap.forEach{
            mul *=it.value
        }
        return mul
    }
}

```

```

class RunOp( hashmap: HashMap< Int, Int>, public var option: Int): HashMapOps(hashmap) {
    var rezultat: Int = 0
    var mutex = Mutex()

```

var counterContext = newSingleThreadContext("CounterContext") **//pentru fiecare obiect al clasei RunOp, se definește un context nou care va rula pe un fir separat.**
newSingleThreadContext("CounterContext") creează un fir de execuție dedicat pentru operațiile efectuate de acest obiect.

```

suspend fun runOperation() = runBlocking {
    withContext(counterContext){
        mutex.withLock {
            if (option == 1){
                rezultat = sum()
            }
            else if( option == 2){
                rezultat = mul()
            }
        }
    }
}
}

```

```

fun main(args: Array<String>) = runBlocking {
    println("Incepem programul!")

    var operation1= RunOp(hashMapOf(Pair(1,1), Pair(2,2), Pair(3,3),Pair(4,4)) ,1)
    var operation2= RunOp(hashMapOf(Pair(1,1), Pair(2,2), Pair(3,3)), 2)

    operation1.runOperation()
    operation2.runOperation()
}

```

```
println(operation1.rezultat)
println(operation2.rezultat)
}
```

- **Python:** Utilizand fabrica de fabrici sa se creeze un program Python care va afisa in functie de apel in mod grafic un patrat un dreptunghi sau un cerc. Se vor desena diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```
import tkinter as tk
from tkinter import *
from abc import ABC, abstractmethod

class FormaGeometrica(ABC):
    @abstractmethod
    def deseneaza(self, canvas):
        pass

class Cerc(FormaGeometrica):
    def __init__(self, x, y, r):
        self.x = x
        self.y = y
        self.r = r

    def deseneaza(self, canvas):
        canvas.create_oval(self.x - self.r, self.y - self.r, self.x + self.r, self.y + self.r, outline='black')

class Triunghi(FormaGeometrica):
    def __init__(self, x1, y1, x2, y2, x3, y3):
        self.x1, self.y1 = x1, y1
        self.x2, self.y2 = x2, y2
        self.x3, self.y3 = x3, y3

    def deseneaza(self, canvas):
        canvas.create_polygon([self.x1, self.y1, self.x2, self.y2, self.x3, self.y3], outline='black')

class Dreptunghi(FormaGeometrica):
    def __init__(self, x1, y1, x2, y2):
        self.x1, self.y1 = x1, y1
        self.x2, self.y2 = x2, y2
```

```

def deseneaza(self, canvas):
    canvas.create_rectangle(self.x1, self.y1, self.x2, self.y2, outline='black')

class Factory(ABC):
    @abstractmethod
    def make(self, type: str):
        pass

class FormaFactory(Factory):
    def make(self, type: str):
        if type == "Dreptunghi":
            return Dreptunghi(50,50, 200, 150)
        elif type == "Cerc":
            return Cerc(300, 200, 50)
        elif type == "Triunghi":
            return Triunghi(100, 150, 150, 50, 200, 150)
        else:
            return None

if __name__ == '__main__':
    root = Tk()
    root.title= ("Desenare forme")
    canvas = tk.Canvas(root, width= 400, height=400)
    canvas.pack()

    fabrica = FormaFactory()

    forma1= fabrica.make("Cerc")
    forma2=fabrica.make("Dreptunghi")
    forma3= fabrica.make("Triunghi")

    forma1.deseneaza(canvas)
    forma2.deseneaza(canvas)
    forma3.deseneaza(canvas)

    root.mainloop()

```

SV 107

- **Kotlin:** Sa se scrie un program Kotlin care va crea echivalentul unei bariere (si apoi va utiliza intr-un exemplu simplu de genul sume de valori dintr-un hasmap realizate pe mai multe fire Java.) Implementarea acestei bariere se va baza pe fire java (apelate din programul Kotlin) si pe mecanismele wait, notify si notify all.

```

package com.pp.laborator
import java.util.*

class Bariera( private val maxThreads: Int) {

    private val lock= java.lang.Object()
    @Volatile private var contorThreads : Int = 0

    fun await() = synchronized(lock)
    {
        contorThreads++
        while ( contorThreads < maxThreads)
        {
            Thread.sleep(1000)
            lock.wait()
        }
        lock.notifyAll()
    }
}

class MyThread( private val threadName: String, private val cb: Bariera,
    private val mutableIterable: MutableIterable<Int>,
        public var sum: Int,
        public var dim: Int,
        public var parte: Int ): Runnable
{
    override fun run (){
        val liste = mutableIterable.chunked(dim)

        sum = liste[parte].sum()
        println("sunt in ${Thread.currentThread().name} ")
        this.cb.await()
    }
}

fun main(args: Array<String>) {
    println(" incepem programul!")

    val bariera: Bariera= Bariera(3)

    val lista = mutableListOf<Int>(10, 20, 30, 40, 50, 60, 70)
    var suma: Int = 0

```

```

var mt1 = MyThread("thread1", bariera, lista, suma, 3, 1)
var mt2 = MyThread("thread2", bariera, lista, suma, 3, 2)
var mt3 = MyThread("thread3", bariera, lista, suma, 3, 2)

var t1: Thread = Thread(mt1)
var t2: Thread = Thread(mt2)
var t3: Thread = Thread(mt3)

t1.start()
t2.start()
t3.start()

t1.join()
t2.join()
t3.join()

var finala = mt1.sum + mt2.sum + mt3.sum
println(finala)
}

```

- **Python:** Utilizand modelul fatada in cadrul unui program Python sa se creeze un obiect de tip boombox(acesta este compus din metode specifice: play, record, fast forward, rewind, record, radio, volume, battery status, etc). Se vor desena diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```

class Player:
    def play(self):
        print("Music is playing")

    def stop(self):
        print("The music stoped")

    def rewind(self):
        print(" We are rewinding the music")

    def fastForward(self):
        print("Music on fastForward")

class Recorder:
    def record(self):
        print("The music is recording")

```

```
class Radio:
    def turnOn(self):
        print("the music is turned on")

    def changeStation(self, station):
        self.station= station
        print("The station is changed")

class VolumeControl:
    def __init__(self):
        self.volume = 5
    def increaseVolume(self):
        print("The volume is increasing")
        self.volume += 1
        print(self.volume)

    def decreaseVolume(self):
        print("The voulume is decreasing")
        self.volume -= 1
        print(self.volume)

class Batery:
    def __init__(self):
        self.batery= 80
    def bateryStatus(self):
        print("The batery status: ")
        print(self.batery)

class BomboxFacade:
    def __init__(self):
        self.player= Player()
        self.recorder= Recorder()
        self.batery= Batery()
        self.radio= Radio()
        self.volume= VolumeControl()

    def play(self):
        self.player.play()

    def stop(self):
        self.player.stop()

    def rewind(self):
```

```

        self.player.rewind()

    def fastForward(self):
        self.player.fastForward()

    def record(self):
        self.recorder.record()

    def baterystatus(self):
        self.batery.bateryStatus()

    def turnOn(self):
        self.radio.turnOn()

    def changeStation(self, station):
        self.radio.changeStation(station)

    def increaseVolume(self):
        self.volume.increaseVolume()

    def decreaseVolume(self):
        self.volume.decreaseVolume()

bombox= BomboxFacade()

bombox.play()
bombox.increaseVolume()
bombox.baterystatus()
bombox.turnOn()
bombox.changeStation("Rock FM")

```

SV 108

- **Kotlin:** Sa se realizeze un program Kotlin care va aplica memoizarea generalizata pentru calculul $f(i) = f(i - 1) + f(i - 2)$ utilizand concurrent hasmap

```

import java.util.concurrent.ConcurrentHashMap

// ConcurrentHashMap pentru memoizare
val cache = ConcurrentHashMap<Int, Int>()

fun fibonacci(n: Int): Int {
    // Verificăm dacă rezultatul este deja în cache

```

```

cache[n]?.let { return it }

// Calculăm valoarea Fibonacci
val result = when (n) {
    0 -> 0
    1 -> 1
    else -> fibonacci(n - 1) + fibonacci(n - 2)
}

// Stocăm rezultatul în cache
cache[n] = result

return result
}

fun main() {
    val n = 10
    val rez = fibonacci(n)
    println("Rezultatul este: $rez")
}

```

- **Python:** Utilizand modelul lant de responsabilitati si nivelul de alerta [0..5] sa se creeze un program Python care va distribui un mesaj(primit la intrare impreuna cu nivelul lui) pentru a fi tratat(afisat) in zona potrivita (e.g 5-paznici muzee, 4-politie, 3-sri, 2-sie, 1-csat, 0-nato). Se vor desena diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```

class baseHandler:
    def __init__(self, next):
        self.nextHandler = next

    def process(self, message, level):
        pass

class MuseumGuargian(baseHandler):
    def process(self, message: str, level: int):
        print(" A fost inregistrata cererea", message)
        if (level == 5):
            print("Gardiabul de muzeu a procesat cereera", message)
        else:
            if (self.nextHandler != None):
                print("Gardianul a pasat cererea")

```

```
self.nextHandler.process(message, level)
```

```
class Police(baseHandler):
```

```
def process(self, message: str, level: int):
```

```
    if(level == 4):
```

```
        print("Politia a procesat cererea", message)
```

```
    else:
```

```
        if(self.nextHandler != None):
```

```
            print("Politia a pasat cererea")
```

```
            self.nextHandler.process(message, level)
```

```
class SRI(baseHandler):
```

```
def process(self, message:str, level:int):
```

```
    if( level == 3):
```

```
        print("SRI a procesat cererea", message)
```

```
    else:
```

```
        if( self.nextHandler != None):
```

```
            print("SRI a pasat cererea")
```

```
            self.nextHandler.process(message, level)
```

```
class SIE(baseHandler):
```

```
def process(self, message:str, level:int):
```

```
    if (level == 2):
```

```
        print("SIE a procesat cererea", message)
```

```
    else:
```

```
        if(self.nextHandler != None):
```

```
            print("SIE a pasat cererea")
```

```
            self.nextHandler.process(message, level)
```

```
class CSAT(baseHandler):
```

```
def process(self, message:str, level:int):
```

```
    if (level == 1):
```

```
        print("CSAT a procesat cerera", message)
```

```
    else:
```

```
        if(self.nextHandler != None):
```

```
            print("CSAT a pasat cererea")
```

```
            self.nextHandler.process(message,level)
```

```
class NATO(baseHandler):
```

```
def process(self, message: str, level:int):
```

```
    if (level == 0):
```

```

    print("Nato a procesat cererea", message)
else:
    print("Nato nu a putut rezolva. Cerere invalida")

```

```

if __name__ == '__main__':
    aparare1= NATO(None)
    aparare2 = CSAT(aparare1)
    aparare3= SIE(aparare2)
    aparare3bis= SRI(aparare3)
    aparare4=Police(aparare3bis)
    aparare5 = MuseumGuargian(aparare4)

    aparare5.process("Atac cu mitraliere la muzeu", 0)

```

SUBIECT CODRINA:

- **Kotlin:** Utilizand modelul prototip pentru niveluri de semnale [0..5] sa se creeze un program Kotlin care va distribui un mesaj(primit la intrare impreuna cu nivelul lui) pentru a fi tratat(afisat) in zona potrivita (e.g 5-paznici muzee, 4-politie, 3-sri, 2-sie, 1-csat, 0-nato). Se vor desena diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```

//definim interfata care va fi implementata de toate clasele
interface MessageHandler {
    fun handleMessage(mesaj: String, level: Int)
}

class Nato: MessageHandler {
    override fun handleMessage(mesaj: String, level: Int) {
        if( level == 0){
            print("NATO a receptionat mesajul : $mesaj")
        }
    }
}

class CSAT: MessageHandler{
    override fun handleMessage(mesaj: String, level: Int) {
        if ( level == 1){
            print( "CSAT a receptionat mesajul: $mesaj")
        }
    }
}

```

```

    }
}

class SIE: MessageHandler{
    override fun handleMessage(mesaj: String, level: Int) {
        if ( level == 2){
            print("SIE a receptionat mesajul: $mesaj")
        }
    }
}

class SRI: MessageHandler{
    override fun handleMessage(mesaj: String, level: Int) {
        if (level == 3){
            print("SRI a receptionat mesajul: $mesaj")
        }
    }
}

class Politie: MessageHandler{
    override fun handleMessage(mesaj: String, level: Int) {
        if ( level == 4){
            print("Politia a receptionat mesajul : $mesaj")
        }
    }
}

class PazniciMuzee: MessageHandler{
    override fun handleMessage(mesaj: String, level: Int) {
        if ( level == 5){
            print("Paznicii de muzee au receptionat mesajul: $mesaj")
        }
    }
}

fun main(){
    val messageHandlers: List< MessageHandler> = listOf(
        Nato(),
        CSAT(),
        SIE(),
        SRI(),
        Politie(),
        PazniciMuzee())
}

```

```

val level =4
val mesaj =" Mesaj"

for (handler in messageHandlers) {
    handler.handleMessage(mesaj, level)
    //fiecare handler este apelat incercand sa gestionam mesajul cu nivelul dat
}

}

```

- **Python:** Sa se implementeze un program ce va instantia obiecte de tip casetofon simplu, boom casetofon, procasetofon si sa poata fi duplicate. Se vor folosi principiile SOLID

```

import copy
from abc import ABC, abstractmethod

class Casetofon(ABC):
    @abstractmethod
    def play_music(self):
        pass

    @abstractmethod
    def duplicate(self):
        pass

class CasetofonSimplu(Casetofon):
    def play_music(self):
        print("Muzica redata de casetofonul simplu")

    def duplicate(self):
        return copy.deepcopy(self)

class BoomCasetofon(Casetofon):
    def play_music(self):
        print("Muzica redata de BoomCasetofon")

    def duplicate(self):
        return copy.deepcopy(self)

```

```

class ProCasetofon(Casetofon):
    def play_music(self):
        print("Muzica redada de ProCasetofon")

    def duplicate(self):
        return copy.deepcopy(self)

def play_music_from_casetofn(casetofon: Casetofon):
    casetofon.play_music()

if __name__ == '__main__':
    casetofon_simplu= CasetofonSimplu()
    casetofon_boom = BoomCasetofon()
    casetofon_pro = ProCasetofon()

    casetofon_simplu.play_music()
    casetofon_pro.play_music()
    casetofon_boom.play_music()

    cb= casetofon_boom.duplicate()
    cp= casetofon_pro.duplicate()
    cs= casetofon_simplu.duplicate()

    print("\n*****")

    cb.play_music()
    cs.play_music()
    cp.play_music()

```

Subiect Tudor:

- **Kotlin:** Observer: clasa Numarator. La schimbarea valorii, numaratorul genereaza un eveniment CounterEvent. Vor exista mai multe corutine in care Numaratorul va fi apelat. Acest eveniment va fi receptat de clasa Receptor care la aparitia evenimentelor afiseaza valoarea numaratorului. Se va desena diagrama de clase si se vor respecta principiile SOLID.

```

package com.pp.laborator

import kotlinx.coroutines.*
import kotlin.properties.Delegates

```

```

// Interfața Observer care va fi implementată de Receptor
interface Observer {
    fun update(newValue: Int)
}

// Clasa care generează evenimentul CounterEvent
class Numarator {

    // Lista de Observers care vor fi notificate la schimbarea valorii
    private val observers = mutableListOf<Observer>()

    // Valoarea contorului
    var value: Int by Delegates.observable(0) { _, oldValue, newValue ->
        if (oldValue != newValue) {
            notifyObservers(newValue)
        }
    }

    // Metoda care adaugă un nou Observer
    fun addObserver(observer: Observer) {
        observers.add(observer)
    }

    // Metoda care notifică toți observatorii despre schimbarea valorii
    private fun notifyObservers(newValue: Int) {
        observers.forEach { it.update(newValue) }
    }

    // Metodă pentru a incrementa valoarea contorului
    fun increment() {
        value++
    }
}

// Clasa care observă schimbările din Numarator și afișează noua valoare
class Receptor : Observer {
    override fun update(newValue: Int) {
        println("Noua valoare a contorului este: $newValue")
    }
}

// Funcția care simulează corutinele și incrementează numaratorul
fun main() = runBlocking {

```

```

val numarator = Numarator()
val receptor1 = Receptor()
val receptor2 = Receptor()

// Adăugăm observatorii la numarator
numarator.addObserver(receptor1)
numarator.addObserver(receptor2)

// Creăm mai multe corutine care vor modifica valoarea numaratorului
val jobs = List(5) {
    launch {
        repeat(5) {
            delay(100L) // Așteptare pentru a simula o operațiune
            numarator.increment()
        }
    }
}

// Așteptăm ca toate corutinele să își termine execuția
jobs.forEach { it.join() }
}

```

- **Python:** Având la intrare un fișier txt, utilizând threading și împartirea în mai multe bucati să se realizeze numărarea cuvintelor din el. Se va folosi funcția care primește un iterator și o constantă $k = 1$ și returnează rezultatul funcției. $f(x,k) = x + k$ peste fiecare element iterator, x este contor. Se utilizează funcția “partial”. La sfârșit va fi generat noul fișier

```

import threading
from functools import partial

def f(x, k):
    return x + k

def countWords(text, k):
    words = text.split()
    return f(len(words), k) // fol fct f pt a adăuga nr-ul de cuv parametrului k

def processChunk(chunk, k, result, index):
    result[index] = countWords(chunk, k) //salveaza nr-ul de cuv din text si le salveaza in lista result

```

```

def main(input_file, output_file, k=1, num_thraeds=4):
    with open('input.txt', 'r') as file:
        text= file.read()

    chunk_size= len(text) // num_thraeds //calc lung fiecarei bucati impartind lung text la nr fire
    chunks = [text[i:i + chunk_size] for i in range(0, len(text),chunk_size)] //impartim textul in bucati
egale de dim chunk_size

    if len(chunks) > num_thraeds: //daca numarul de bucati e mai mare decat nr fire
        chunks[-2] += chunks[-1] //ultima buc se lipeste la penultima
        chunks = chunks[:-1] //ultima e eliminata

    results = [0] * len(chunks) //initializam lista

    threads = []

    for i in range(len(chunks)):
        thread= threading.Thread(target= processChunck, args=(chunks[i], k, results,i))
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    total_words= sum(results)

    with open(output_file, 'w') as file:
        file.write(f"Total words (with k={k}): {total_words}\n")

if __name__ == '__main__':
    main('input.txt', 'output.txt', k=1, num_thraeds=4)

```

SV 70

- **Python:** Sa se scrie un program Python utilizand asyncio si futures care aplica o procesare pipeline asupra elementelor dintr-un dictionar (index, valoare); prima corutina inmulteste elementele cu o constanta, a doua corutina le aduna doua cate doua, iar rezultatul este suprascris in locul primului element din adunare iar a treia corutina inlocuieste cu zero elementele pare din dictionar

```

import asyncio

async def multiply_by_ct(data, constant):
    for key in data:
        data[key] *= constant
        await asyncio.sleep(0)
    return data

async def add_pairs(data):
    keys= list(data.keys())
    for i in range(0, len(keys) -1 , 2): #itereaza prin key 2 cate 2
        data[keys[i]] = data[keys[i]] + data[keys[i+1]]
        await asyncio.sleep(0)
    return data

async def replace_even_with_zeros(data):
    for key in data:
        if data[key] % 2 == 0:
            data[key] = 0
        await asyncio.sleep(0)
    return data

async def data_process_pipeline(data, constant):
    #inmultire cu o ct
    data = await multiply_by_ct(data, constant)
    print(f"After multiplication: {data}")
    #adunare 2 cate 2
    data = await add_pairs(data)
    print(f"After adding pairs: {data}")
    #inlocuirea elem pare cu 0
    data = await replace_even_with_zeros(data)
    print(f"After replacing with zeros: {data}")

    return data

async def main():
    data = {0:1, 1:2, 2:3, 3:4, 4: 5}
    constant=3
    result= await data_process_pipeline(data, 3)
    print(f"final result: {result}")

asyncio.run(main())

```

SV 65

- **Python:** Sa se scrie un program Python utiiland multiprocessing care distribuie intr-o maniera ciclica un cuvand dintr-un fisier catre alte trei procese si fiecare din acestea afiseaza numele lui si cuvantul primit la consola. Comunicarea intre procese va fi realizata utilizand cozi.

```
import multiprocessing
import time

def worker(name, queue):
    while True:
        word = queue.get()
        if word is None:
            break
        print(f'{name} received: {word}')
        time.sleep(1)

def main():
    nrProcese= 3
    processes =[]
    queue = multiprocessing.Queue()

    #cream si pornim procesele
    for i in range(nrProcese):
        p= multiprocessing.Process(target=worker, args=(f'Process-{i+1}', queue))
        p.start()
        processes.append(p)

    #citim cuv din fis si le adaugam in coada
    with open('words.txt', 'r') as file:
        words =[line.strip() for line in file ]

    #distribuim cuvintele in mod ciclic catre procese
    for i, word in enumerate(words):
        queue.put(word)

    #semnalam procesele sa se opreasca
    for _ in range(nrProcese):
        queue.put(None)

    #Asteptam sa se termine procesele
    for p in processes:
```

```
p.join()
```

```
if __name__ == '__main__':  
    main()
```

SV 38

- **Kotlin:** Utilizandf ADT- uri (eventual lambda) si transformari specifice colectiilor avand la intrare multimea $A = \{1, 2, \dots, 100\}$. Sa se scrie un program Kotlin care determina numarul submultimilor cu 4 elemente ale multimii A, care contine elementul 1(componenta numarului)

```
fun main() {  
    val A=(1..100).toList()  
    val submultimeDe4= combinations(A, 4)  
    val submultimiCu1= submultimeDe4.filter { it.contains(1)}  
    println("Numarul este de ${submultimiCu1.size}")  
}  
  
fun <T> combinations(list: List<T>, k: Int): List<List<T>> {  
    if ( k== 0) return listOf(emptyList())  
    if (list.isEmpty()) return emptyList()  
    val head = list.first()  
    val tail = list.drop(1)  
    val combsWithHead = combinations(tail, k -1).map { listOf(head) + it}  
    val combsWithoutHead = combinations( tail, k)  
    return combsWithHead+ combsWithoutHead  
}
```

- **Python:** Utilizand tkinter si functii lambda sa se creeze un program Python care va afisa in eticheta ferestrei operatiile realizate cu mouse ul in momentul in care facem un desen simplu (de ex trag un triunghi din linii) pe un canvas din acea fereastră. Programul va avea meniu

```
import tkinter as tk  
from tkinter import messagebox, colorchooser
```

```

def start_draw(event):
    global start_x, start_y
    start_x, start_y = event.x, event.y
    canvas.bind("<B1-Motion>", draw_line)

def draw_line(event):
    global start_x, start_y
    canvas.create_line(start_x, start_y, event.x, event.y, fill=current_color, width=2)
    # Update the label with the current operation
    operations_label.config(text=f"Drawing line from ({start_x}, {start_y}) to ({event.x}, {event.y})")
    start_x, start_y = event.x, event.y

def clear_canvas():
    canvas.delete("all")
    operations_label.config(text="Canvas cleared")

def choose_color():
    global current_color
    color = colorchooser.askcolor()[1]
    if color:
        current_color = color
        color_button.config(bg=current_color)

def show_about():
    messagebox.showinfo("About", "This is a simple drawing app using Tkinter.")

# Create the main window
root = tk.Tk()
root.title("Drawing App")

# Initialize color
current_color = "black"

# Create a menu
menu = tk.Menu(root)
root.config(menu=menu)

file_menu = tk.Menu(menu, tearoff=0)
menu.add_cascade(label="File", menu=file_menu)
file_menu.add_command(label="Clear", command=clear_canvas)
file_menu.add_separator()
file_menu.add_command(label="Exit", command=root.quit)

help_menu = tk.Menu(menu, tearoff=0)

```

```

menu.add_cascade(label="Help", menu=help_menu)
help_menu.add_command(label="About", command=show_about)

# Create a canvas and an operations label
canvas = tk.Canvas(root, width=500, height=400, bg="white")
canvas.pack(fill=tk.BOTH, expand=True)

operations_label = tk.Label(root, text="Draw something on the canvas", bg="lightgrey")
operations_label.pack(fill=tk.X)

# Create a color button
color_button = tk.Button(root, text="Choose Color", command=choose_color, bg=current_color)
color_button.pack(pady=5)

# Bind the mouse events to the canvas
canvas.bind("<Button-1>", start_draw)

# Run the application
root.mainloop()

```

SV 44

- **Kotlin:** Sa se scrie un program Kotlin care avand la intrare doua multimi A si B de cate 15 elemente initializate aleator care vor fi depuse in niste colectii sa se aplice utilizand calcului lambda si operatorii specifici produsul cartezian $A \times B$ iar rezultatul la randul lui depus intr-o colectie si aceasta afisata

```

import kotlin.random.Random

fun main() {
    val A = List(15) { Random.nextInt(1, 101)}
    val B = List(15){Random.nextInt(1, 1001)}

    println("Multimea A: $A")
    println("Multimea B: $B")

    val AxB = A.flatMap{ a -> B.map{ b -> a to b }}

    println("Produsul cartezian: $AxB")
    println("*****")
    AxB.forEach{ println(it)}
}

```

- **Python:** Sa se creeze o aplicatie Python pentru rezervarea unei camere de hotel. Camerele (cu attributele si operatorii lor) sunt obiecte si sunt retinute cu un ADT. Cu ajutorul unor decoratori peste functiile de baza se vor implementa un flux suplimentar de tratare a platii urmatorului serviciu suplimentar: consumul din barul camerei cand se genereaza nota de plata. Se vor prezenta si diagramele de clase si obiecte.

```
from functools import wraps

class Camera:
    def __init__(self, numar, tip, pret):
        self.numar= numar
        self.tip= tip
        self.pret= pret
        self.bar_consum= 0 #consumul din barul camerei

    def __str__(self):
        return f"Camera {self.numar}: {self.tip}, pret: {self.pret} RON"

class Rezervare:
    def __init__(self, camera, nume_client, zile):
        self.camera= camera
        self.nume_client = nume_client
        self.zile= zile
        self.pret_total = self.calcul_pret_total()

    def calcul_pret_total(self):
        return self.camera.pret * self.zile + self.camera.bar_consum

    def __str__(self):
        return f"Rezervare pt {self.nume_client} la {self.camera} pentru {self.zile} zile. Pret Total: {self.calcul_pret_total()} RON"

def cu_bar_decorator(func):
    @wraps(func)
    def wrapper(rezervare, *args, **kwargs):
        #calculam pretul folosind functia originala
        pret_total = func(rezervare, *args, **kwargs)
        pret_total += rezervare.camera.bar_consum
```

```

        return pret_total
    return wrapper

class Rezervare:
    def __init__(self, camera, nume_client, zile):
        self.camera= camera
        self.nume_client= nume_client
        self.zile= zile

    @cu_bar_decorator
    def calcul_pret_total(self):
        return self.camera.preeliza * self.zile

    def __str__(self):
        return f'Rezervare pentru {self.nume_client} la {self.camera} pentru {self.zile}. Pret total: {self.calcul_pret_total} RON'

#exemplu de utilizare
camera = Camera(numar= 101, tip="Single", pret=200)
rezervare = Rezervare(camera=camera, nume_client="Ion Popescu", zile=3)
camera.bar_consum = 50
print(rezervare)

```

Subiect restanta1

- **Kotlin:** Sa se utilizeze o implementare a interfetei persoana pt o agenta si o clasa utilizator de tip delegat. Programul Kotlin va afisa numele utilizatorului in urma unei cautari prin agenda. Se va desena diagrama UML si se vor explica principiile SOLID.

```

interface Persoana {
    val nume: String
}

class Agenda( private val persoane: List<Persoana>){
    fun cautaPersoane(nume:String): Persoana? {
        return persoane.find {it.nume == nume}
    }
}

```

```

class PersoanaImpl( override val nume: String): Persoana

class Utilizator( persoana: Persoana): Persoana by persoana

fun main(){
    val Utilizatori = listOf (
        Utilizator(PersoanaImpl("Ion Popescu")),
        Utilizator(PersoanaImpl("Maria Ionescu")),
        Utilizator(PersoanaImpl("George Enescu"))
    )

    val agenda= Agenda(Utilizatori)
    val numeCautat= "Maria Ionescu"
    val rezultat= agenda.cautaPersoane(numeCautat)

    if (rezultat != null){
        println("Persoana ${rezultat.nume} a fost gasita")
    }else
    {
        println("Persoana nu a fost gasita")
    }
}

```

Subiect restanta2 (ST31) (HABAR NU AM CE E CU CERINTA ASTA, CODUL E DE PE CHATGBT)

- **Kotlin:** Sa se scrie un program Kotlin care preia un fisier text (scris de programator) la intrare organizata cuvintele intr-un arbore binar utilizand o constrangere la alegere iar apoi cu ajutorul unei clase specifice si al functorilor va afisa portiuni din arbore.

```

import java.io.File

// Structura unui nod din arborele binar
data class TreeNode(val word: String, var left: TreeNode? = null, var right: TreeNode? = null)

// Clasa care gestionează arborele binar
class BinaryTree {
    var root: TreeNode? = null
}

```

```

// Funcție de inserare bazată pe ordinea lexicografică
fun insert(word: String) {
    root = insertRec(root, word)
}

private fun insertRec(node: TreeNode?, word: String): TreeNode {
    if (node == null) {
        return TreeNode(word)
    }
    // Comparăm cuvintele și decidem unde inserăm noul cuvânt
    if (word < node.word) {
        node.left = insertRec(node.left, word)
    } else if (word > node.word) {
        node.right = insertRec(node.right, word)
    }
    return node
}

// Functor pentru a aplica o funcție pe fiecare nod al arborelui
fun traverseAndApply(node: TreeNode? = root, action: (String) -> Unit) {
    if (node != null) {
        traverseAndApply(node.left, action)
        action(node.word)
        traverseAndApply(node.right, action)
    }
}

// Citim fișierul și populăm arborele
fun readWordsFromFile(fileName: String): List<String> {
    return File(fileName).readText()
        .split("\\s+").toRegex() // Separăm pe baza spațiilor (orice secvență de spații)
        .filter { it.isNotBlank() } // Eliminăm cuvintele goale
}

fun main() {
    val fileName = "input.txt" // Fișierul de intrare
    val words = readWordsFromFile(fileName) // Citim cuvintele din fișier

    // Creăm arborele binar și adăugăm cuvintele
    val tree = BinaryTree()
    for (word in words) {
        tree.insert(word)
    }
}

```

```

// Afișăm cuvintele în ordine lexicografică folosind un functor
println("Cuvintele din arbore în ordine lexicografică:")
tree.traverseAndApply { word ->
    println(word)
}

// Afișăm doar cuvintele care încep cu litera "a" (ca exemplu de filtrare)
println("\nCuvintele care încep cu litera 'a':")
tree.traverseAndApply { word ->
    if (word.startsWith("a", ignoreCase = true)) {
        println(word)
    }
}
}

```

SV 26

- **Python:** Fie înregistrarea = { 'Nume': 'Bula', 'Locatia': { 'Oras': 'Pocreaca', 'Tara': 'RO' }, 'Gen': 'Feminin' }. Sa se realizeze un program Python care va aplica data flatten peste acest rezultat.

```

def flatten_dict(d, parent_key= "", sep=""):
    items=[]
    for k,v in d.items():
        new_key= parent_key + sep + k if parent_key else k
        if isinstance(v, dict):
            items.extend(flatten_dict(v, new_key, sep=sep).items())
        else:
            items.append((new_key, v))
    return dict(items)

inregistrare= {
    'Nume': 'Bula',
    'Locatia': {
        'Oras': 'Pocreaca',
        'Tara': 'RO',
    },
    'Gen': 'Feminin'
}

```

```
flat_inregistrare= flatten_dict(inregistrare)

print(flat_inregistrare)
```

SV 58

- **Kotlin:** Utilizand Kotlin si OPP sa se creeze (KBD sau din program) un grup de oameni care pot efectua (cu totii) urmatoarele activitati: mananca, beau, danseaza. Fiecare om este diferit deci functiile vor primi parametri diferiti. Pornind de la urmatoarele declaratii sa se creeze un program care raspunde la intrebari: ion mananca mere, bea bere si danseaza cu femei. Vasile mananca pere, bea vodka si danseaza cu barbati. Alex bea vin mananca prajituri brune si danseaza cu sefii. Exemplu de intrebare: Cu cine poate dansa Vasile si ce poat manca barbatii dar ce pot manca femeile? Se va folosi polimorfismul. Se va desena diagrama UML (clasa si obiecte). Se va explica maniera de aplicare a principiilor SOLID.

```
// Interfete pentru activități comune (manâncă, bea, dansează)
interface Mananca {
    fun mananca(aliment: String)
}

interface Bea {
    fun bea(bautura: String)
}

interface Danseaza {
    fun danseaza(partener: String)
}

// Clasa abstractă Om care implementează aceste interfețe
abstract class Om : Mananca, Bea, Danseaza {
    abstract val nume: String
}

// Clasa pentru Ion care extinde clasa abstractă Om
class Ion : Om() {
    override val nume = "Ion"

    override fun mananca(aliment: String) {
        println("$nume mananca $aliment.")
    }
}
```

```

    }

    override fun bea(bautura: String) {
        println("$nume bea $bautura.")
    }

    override fun danseaza(partener: String) {
        println("$nume danseaza cu $partener.")
    }
}

// Clasa pentru Vasile care extinde clasa abstractă Om
class Vasile : Om() {
    override val nume = "Vasile"

    override fun mananca(aliment: String) {
        println("$nume mananca $aliment.")
    }

    override fun bea(bautura: String) {
        println("$nume bea $bautura.")
    }

    override fun danseaza(partener: String) {
        println("$nume danseaza cu $partener.")
    }
}

// Clasa pentru Alex care extinde clasa abstractă Om
class Alex : Om() {
    override val nume = "Alex"

    override fun mananca(aliment: String) {
        println("$nume mananca $aliment.")
    }

    override fun bea(bautura: String) {
        println("$nume bea $bautura.")
    }

    override fun danseaza(partener: String) {
        println("$nume danseaza cu $partener.")
    }
}

```

```
// Funcționalitatea principală a programului
fun main() {
    val ion = Ion()
    val vasile = Vasile()
    val alex = Alex()

    // Ion mananca mere, bea bere si danseaza cu femei.
    ion.mananca("mere")
    ion.bea("bere")
    ion.danseaza("femei")

    // Vasile mananca pere, bea vodka si danseaza cu barbati.
    vasile.mananca("pere")
    vasile.bea("vodka")
    vasile.danseaza("barbati")

    // Alex mananca prajituri brune, bea vin si danseaza cu sefi.
    alex.mananca("prajituri brune")
    alex.bea("vin")
    alex.danseaza("sefi")

    // Răspuns la întrebări:
    // Cu cine poate dansa Vasile?
    println("Vasile poate dansa cu barbati.")

    // Ce pot manca barbatii?
    println("Barbatii pot manca pere.")

    // Ce pot manca femeile?
    println("Femeile pot manca prajituri brune si mere.")
}
```

- **Python:** Sa se scrie un program Python (utilizand threading) care va utiliza subclase active (cu un fir) pentru procesarea simultana a unui hasmap bazata pe functii pure. Clasa de baza va stabili operatiile (de ex adunare, scadere, inmultire binara pe 16 biti) iar subclasele vor pune la dispozitie obiecte care se executa in fire separate..Se va prezenta diagrama de clase si de obiecte..Se vor utiliza semafoare

```
import threading
```

```

import random

# Clasa de bază pentru operații
class Operation:
    def __init__(self, hashmap, semaphore):
        self.hashmap = hashmap
        self.semaphore = semaphore

    def execute(self):
        raise NotImplementedError("Subclasa trebuie să implementeze metoda execute.")

# Subclasa pentru adunare
class Addition(Operation):
    def execute(self):
        with self.semaphore:
            for key in self.hashmap:
                self.hashmap[key] += 1
            print(f"Adunare: {key} -> {self.hashmap[key]}")

# Subclasa pentru scădere
class Subtraction(Operation):
    def execute(self):
        with self.semaphore:
            for key in self.hashmap:
                self.hashmap[key] -= 1
            print(f"Scădere: {key} -> {self.hashmap[key]}")

# Subclasa pentru înmulțire binară pe 16 biți
class BinaryMultiplication(Operation):
    def execute(self):
        with self.semaphore:
            for key in self.hashmap:
                self.hashmap[key] = (self.hashmap[key] * 2) & 0xFFFF
            print(f"Înmulțire binară: {key} -> {self.hashmap[key]}")

# Funcția principală
def main():
    # Hashmap inițial
    hashmap = {f"key{i}": random.randint(0, 100) for i in range(5)}
    print("Hashmap inițial:", hashmap)

    # Semafoare
    semaphore = threading.Semaphore(1)

```

```

# Crearea obiectelor de operație
addition = Addition(hashmap, semaphore)
subtraction = Subtraction(hashmap, semaphore)
multiplication = BinaryMultiplication(hashmap, semaphore)

# Crearea firelor
threads = [
    threading.Thread(target=addition.execute),
    threading.Thread(target=subtraction.execute),
    threading.Thread(target=multiplication.execute)
]

# Pornirea firelor
for thread in threads:
    thread.start()

# Așteptarea finalizării firelor
for thread in threads:
    thread.join()

print("Hashmap final:", hashmap)

if __name__ == "__main__":
    main()

```

SV 93

- **Kotlin:** Sa se scrie un program Kotlin care va utiliza subclase active (cu actori) pt procesarea simultana a unui hashmap bazata. Clasa de baza va stabili operatiile (de ex adunare, scadere, inmultire, impartire) iar subclasele vor pune la dispozitie obiecte care executa in fire separate. Se va prezenta diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```

import kotlinx.coroutines.*
import kotlinx.coroutines.channels.actor

// Clasa de bază pentru operații matematice
sealed class MathOperation
data class Add(val a: Int, val b: Int) : MathOperation()
data class Subtract(val a: Int, val b: Int) : MathOperation()
data class Multiply(val a: Int, val b: Int) : MathOperation()
data class Divide(val a: Int, val b: Int) : MathOperation()

```

```

// Actorul de bază pentru procesarea operațiilor matematice
fun CoroutineScope.mathActor() = actor<MathOperation> {
    for (operation in channel) {
        when (operation) {
            is Add -> println("Adunare: ${operation.a} + ${operation.b} = ${operation.a + operation.b}")
            is Subtract -> println("Scădere: ${operation.a} - ${operation.b} = ${operation.a -
operation.b}")
            is Multiply -> println("Înmulțire: ${operation.a} * ${operation.b} = ${operation.a *
operation.b}")
            is Divide -> {
                if (operation.b != 0) {
                    println("Împărțire: ${operation.a} / ${operation.b} = ${operation.a / operation.b}")
                } else {
                    println("Împărțire: nu se poate împărți la 0!")
                }
            }
        }
    }
}

// Funcția principală
fun main() = runBlocking {
    val hashMap = hashMapOf(
        "adunare" to Add(10, 20),
        "scadere" to Subtract(30, 10),
        "inmultire" to Multiply(5, 6),
        "impartire" to Divide(40, 5),
        "impartire_zero" to Divide(10, 0)
    )

    // Creăm actorul
    val mathProcessor = mathActor()

    // Procesăm HashMap-ul folosind actorul
    for (operation in hashMap.values) {
        mathProcessor.send(operation)
    }

    // Oprirea actorului
    mathProcessor.close()
}

```

- **Python:** utilizand modelul prototip sa se deseneze in mod grafic utilizand Python mai multe carucioare identice plasate in diverse pozitii. Se vor desena diagrama de clase si de obiecte. Se va explica maniera de aplicare a principiilor SOLID.

```
import tkinter as tk

class Carucior:
    def __init__(self, x, y, width=100, height=50):
        self.x = x
        self.y = y
        self.width = width
        self.height = height

class DesenatorCarucioare:
    def __init__(self, canvas):
        self.canvas = canvas

    def deseneaza_carucior(self, carucior):
        self.canvas.create_rectangle(carucior.x, carucior.y,
                                     carucior.x + carucior.width,
                                     carucior.y + carucior.height,
                                     outline="red", fill="")

    def deseneaza_carucioare(self, carucioare):
        for carucior in carucioare:
            self.deseneaza_carucior(carucior)

# Configurarea ferestrei Tkinter
root = tk.Tk()
root.title("Desenare Cărucioare")
canvas = tk.Canvas(root, width=400, height=400)
canvas.pack()

# Crearea mai multor carucioare
carucior1 = Carucior(50, 50)
carucior2 = Carucior(150, 100)
carucior3 = Carucior(250, 150)

# Crearea desenatorului și desenarea carucioarelor
desenator = DesenatorCarucioare(canvas)
desenator.deseneaza_carucioare([carucior1, carucior2, carucior3])
```

```
# Rularea ferestrei Tkinter
root.mainloop()
```

- **Kotlin:** Utilizand o fabrica abstracta sa se scrie un program Kotlin care permite crearea la cerere a trei obiecte (student, profesor, secretar) acestea la randul lor permit crearea de obiecte specifici (la profesori cream direct prof, conf, sl, as, la student: integralist, restantier si repetent. La secretar avem secretar sef si secretar I si II)

```
// Obiecte de bază: Student, Profesor, Secretar
interface Student {
    fun descriere()
}

interface Profesor {
    fun descriere()
}

interface Secretar {
    fun descriere()
}

// Implementări concrete pentru Student
class Integralist : Student {
    override fun descriere() = println("Student Integralist")
}

class Restantier : Student {
    override fun descriere() = println("Student Restantier")
}

class Repetent : Student {
    override fun descriere() = println("Student Repetent")
}

// Implementări concrete pentru Profesor
class ProfesorTitular : Profesor {
    override fun descriere() = println("Profesor Titular")
}

class ProfesorConferentiar : Profesor {
```

```

    override fun descriere() = println("Profesor Conferențiar")
}

class SefLucrari : Profesor {
    override fun descriere() = println("Șef Lucrări")
}

class Asistent : Profesor {
    override fun descriere() = println("Asistent")
}

// Implementări concrete pentru Secretar
class SecretarSef : Secretar {
    override fun descriere() = println("Secretar Șef")
}

class SecretarI : Secretar {
    override fun descriere() = println("Secretar Grad I")
}

class SecretarII : Secretar {
    override fun descriere() = println("Secretar Grad II")
}

// Fabrica Abstractă
interface AbstractFactory {
    fun createStudent(): Student
    fun createProfesor(): Profesor
    fun createSecretar(): Secretar
}

// Fabrica concretă pentru fiecare tip de obiect
class StudentFactory : AbstractFactory {
    override fun createStudent(): Student = Integralist() // Poți schimba tipul de student
    override fun createProfesor(): Profesor = ProfesorTitular() // Nu se folosește în această fabrică
    override fun createSecretar(): Secretar = SecretarSef() // Nu se folosește în această fabrică
}

class ProfesorFactory : AbstractFactory {
    override fun createStudent(): Student = Integralist() // Nu se folosește în această fabrică
    override fun createProfesor(): Profesor = ProfesorTitular() // Poți schimba tipul de profesor
    override fun createSecretar(): Secretar = SecretarSef() // Nu se folosește în această fabrică
}

```

```

class SecretarFactory : AbstractFactory {
    override fun createStudent(): Student = Integralist() // Nu se folosește în această fabrică
    override fun createProfesor(): Profesor = ProfesorTitular() // Nu se folosește în această fabrică
    override fun createSecretar(): Secretar = SecretarSef() // Poți schimba tipul de secretar
}

// Testare cod
fun main() {
    val studentFactory = StudentFactory()
    val profesorFactory = ProfesorFactory()
    val secretarFactory = SecretarFactory()

    val student = studentFactory.createStudent()
    student.descriere()

    val profesor = profesorFactory.createProfesor()
    profesor.descriere()

    val secretar = secretarFactory.createSecretar()
    secretar.descriere()
}

```

- **Python:** Sa se scrie un program Python (utilizand threading) care va utiliza subclase active(cu un fir) pentru procesarea simultana a unui hasmap bazata pe functii pure. Clasa de baza va stabili operatiile (de ex adunare, scadere, inmultire binara pe 16 biti) iar subclasele vor pune la dispozitie obiecte care se executa in fire separate, Se va utiliza rlock().

```

import threading
import random

# Clasa de bază pentru operații
class Operation(threading.Thread):
    def __init__(self, hashmap, rlock):
        super().__init__()
        self.hashmap = hashmap
        self.rlock = rlock

    def run(self):
        raise NotImplementedError("Subclasa trebuie să implementeze metoda run.")

```

```

# Subclasa pentru adunare
class Addition(Operation):
    def run(self):
        with self.rlock:
            for key in self.hashmap:
                self.hashmap[key] += 1
                print(f"Adunare: {key} -> {self.hashmap[key]}")

# Subclasa pentru scădere
class Subtraction(Operation):
    def run(self):
        with self.rlock:
            for key in self.hashmap:
                self.hashmap[key] -= 1
                print(f"Scădere: {key} -> {self.hashmap[key]}")

# Subclasa pentru înmulțire binară pe 16 biți
class BinaryMultiplication(Operation):
    def run(self):
        with self.rlock:
            for key in self.hashmap:
                self.hashmap[key] = (self.hashmap[key] * 2) & 0xFFFF
                print(f"Înmulțire binară: {key} -> {self.hashmap[key]}")

# Funcția principală
def main():
    # Hashmap inițial
    hashmap = {f"key{i}": random.randint(0, 100) for i in range(5)}
    print("Hashmap inițial:", hashmap)

    # RLock
    rlock = threading.RLock()

    # Crearea obiectelor de operație
    addition = Addition(hashmap, rlock)
    subtraction = Subtraction(hashmap, rlock)
    multiplication = BinaryMultiplication(hashmap, rlock)

    # Pornirea firelor
    addition.start()
    subtraction.start()
    multiplication.start()

```

```
# Așteptarea finalizării firelor
addition.join()
subtraction.join()
multiplication.join()

print("Hashmap final:", hashmap)

if __name__ == "__main__":
    main()
```

PROBLEMA 5 (Python)

Python: Sa se implementeze in python un automat cu trei stari (FSM) in care sa se proceseze cu expresii lambda o lista de numere intregi trimisa ca argument in constructor astfel:
in s0 se verifica daca mai sunt elemente in lista (daca da, se aduce in s1
unde se identifica si se sterge primul element gasit apoi se duce in s2)
in s2 se identifica si se sterge primul element neprin gasit apoi se trece in s0
la fiecare stergere se afiseaza elementul sters
in s0 daca nu mai exista elemente, se anunta acest lucru iar programul se termina

```
class FSM:
    def __init__(self, lista):
        self.lista = lista
        self.stare_curenta = 's0'

    def s0(self):
        if not self.lista:
            print("Nu mai sunt elemente în listă. Programul s-a terminat.")
        else:
            self.stare_curenta = 's1'
            self.s1()

    def s1(self):
        if self.lista:
            element = self.lista.pop(0)
            print(f"Element sters in s1: {element}")
            self.stare_curenta = 's2'
            self.s2()

    def s2(self):
        if self.lista:
            element = self.lista.pop(0)
```

```
print(f"Element sters in s2: {element}")  
self.stare_curenta = 's0'  
self.s0()
```

```
def ruleaza(self):  
    while self.stare_curenta == 's0':  
        self.s0()
```

```
lista = [1, 2, 3, 4, 5, 6]  
automat = FSM(lista)  
automat.ruleaza()
```